AGI Stub 8 Analysis

Rihaan

Contents

1 Introduction

This report studies how the population of high-income tax filers changed across U.S. counties between 2019 and 2021. The IRS Statistics of Income (SOI) data divides every county's filing population into eight Adjusted Gross Income (AGI) brackets, called stubs. Stub 8 is the top bracket: returns reporting AGI of $200,000 or more. Top-bracket earners are the smallest group in number but the largest in fiscal weight, paying roughly 38% of all federal income tax receipts, so changes in where they file have first-order consequences for state and local public finance.

Nationally, the count of stub-8 returns grew by roughly 28% between 2019 and 2021. That two-year jump is far larger than any plausible net migration of high earners, so most of the growth reflects two non-migration channels: bracket creep (income growth pushing existing filers above the $200,000 threshold) and capital-gains realizations from the 2020 to 2021 equity-market rally. A smaller share is genuine residential mobility (the pandemic-era relocation wave to lower-cost or higher-amenity counties). The model in this report does not separate these channels; it predicts the total growth in stub-8 filers per 1,000 residents in each county and identifies which county-level characteristics best explain that growth.

The analytical pipeline has four main stages. First, choose a response variable from three candidates (raw change, percent change, change per 1,000 population) and apply a Box-Cox power transformation to make the residuals approximately normal and homoskedastic. Second, identify and remove two groups of predictors that cannot honestly enter the model: variables that are structurally zero at stub 8 (the EITC family) and variables that are themselves derived from the same year-over-year filer counts that produce the response (the IRS migration family). Third, reduce the remaining ~40 candidate predictors to a top 8 through three thematic best-subsets sweeps and a pooled re-competition, fit the untransformed top-8, and use Cook's distance to trim high-influence outlier counties before going further. Fourth, layer log transformations, quadratic terms, and pairwise interactions on the top 8, keep the significant interactions, and run one final exhaustive best-subsets pass over that feature set to pick the model that maximizes adjusted R-squared.

2 Data Sources

The analysis dataset joins three federal sources at the five-digit county FIPS code:

- IRS Statistics of Income (SOI) county-level tax-return tables, which provide stub-8 return counts and a rich set of income composition, deduction, and tax liability variables computed from individual returns filed in each county and bracket.
- IRS SOI migration data (county-to-county inflow and outflow of returns, individuals, and aggregate AGI).
- U.S. Census Bureau intercensal population estimates and Bureau of Economic Analysis (BEA) per-capita personal income and GDP for 2019 through 2022.

The file used here (`county_1923_May3AGI8_SK.csv`) is already filtered to `agi_stub == 8`, so each row represents one county at the top income bracket. The national sample is roughly 3,100 counties.

One important data property deserves highlighting The income-composition and tax variables (labor share, tax per return, mortgage interest per return, etc.) are observed at the county-by-stub level: they describe stub-8 filers specifically within each county. The demographic variables (working-age share, elderly share, male share) are county-wide Census variables describing all residents regardless of income. This asymmetry between stub-specific tax variables and county-wide demographic proxies is a hard limitation of the SOI data and is revisited in the limitations section.

2.1 Loading and Renaming

The raw column names follow IRS-internal conventions. I rename them once to make every downstream summary easier to read. Two conventions are imposed: `pct_filers_*` for share-of-filers variables (originally suffixed `_incidence`), and the explicit suffixes `_filers` and `_persons` on the migration variables so the unit being counted is always clear.

```
df_raw <- read.csv(data_path, fileEncoding = "cp1252", check.names  = TRUE)

df_raw <- df_raw %>%
  dplyr::select(-any_of(c("stname.1", "ctyname.1", "fips.1")))

df_raw <- df_raw %>% rename(
  population = popestimate, elderly_share = elderly_share_pop,
  transfer_share = transfer_like_share, pct_filers_wages = wage_incidence,
  pct_filers_capgains = capgains_incidence, pct_filers_business = business_incidence,
  pct_filers_rental = rental_incidence,
  pct_filers_unempl = unemployment_incidence,
  pct_filers_mortgage = mortgage_interest_incidence,
  pct_filers_proptax= property_tax_incidence, pct_filers_eitc = eitc_incidence,
  eff_tax_rate_agi = effective_tax_rate_on_agi,
  eff_tax_rate_taxable = effective_tax_rate_on_taxable,
  credits_share = credits_share_of_tax_before,
  mortgage_per_return = mortgage_interest_per_return,
  proptax_per_return = property_tax_per_return,
  salt_income_per_return = salt_income_tax_per_return,
  salt_sales_per_return = salt_sales_tax_per_return,
  refund_ctc_per_return = refundable_ctc_per_return,
  refund_edu_per_return = refundable_edu_per_return,
  refund_eitc_proxy_per_return = refundable_eitc_proxy_per_return,
  eitc_per_claimant = eitc_per_eitc_return,
```

```
  migration_rate_filers = migration_rate_returns,
  migration_rate_persons = migration_rate,
  net_migration_filers = net_migration_returns_not_AGI,
  net_migration_persons = net_migration_not_AGI,
  inflow_filers = inflow_returns, outflow_filers= outflow_returns)
```

```
df <- df_raw %>%
  filter(agi_stub == 8) %>%
  filter(!is.na(Returns19), !is.na(Returns21),
         !is.na(Individuals19), !is.na(Individuals21))

df <- df %>%
  mutate(
    chg_returns_raw = Returns21 - Returns19,
    chg_individuals_raw  = Individuals21- Individuals19,
    chg_returns_per_1k  = (Returns21 - Returns19) / population * 1000,
    chg_individuals_per_1k  = (Individuals21 - Individuals19) / population * 1000,
    pct_chg_returns = ifelse(Returns19 > 0,(Returns21 - Returns19) / Returns19 * 100, NA_real_),
    pct_chg_individuals = ifelse(Individuals19 > 0,
(Individuals21 - Individuals19) / Individuals19 * 100,NA_real_))
```

3 Response Variable

Three candidate response variables were considered. Each represents the same underlying quantity (the change in stub-8 returns between 2019 and 2021) but on a different scale. The boxplots below compare their distributions on the same sample.

```
par(mfrow = c(1, 3), mar = c(4, 4, 3, 1))

boxplot(df$chg_returns_raw,
        main = "Raw change",
        ylab = "Returns21 - Returns19",
        col  = "lightblue")

boxplot(df$pct_chg_returns,
        main = "Percent change",
        ylab = "% change",
        col  = "skyblue")

boxplot(df$chg_returns_per_1k,
        main = "Change per 1,000 pop (selected)",
        ylab = "(Returns21 - Returns19) / pop * 1000",
        col  = "steelblue")
```

```
par(mfrow = c(1, 1))
```

The raw difference is dominated by a few large counties (the maximum is roughly two orders of magnitude above the IQR), so any regression on it would fit county size rather than the drivers of interest. The percent change is unstable when `Returns19` is small: in low-population rural counties even a single-digit change blows up into a huge percentage. The per-1,000-population scaling sits between these two: it removes the

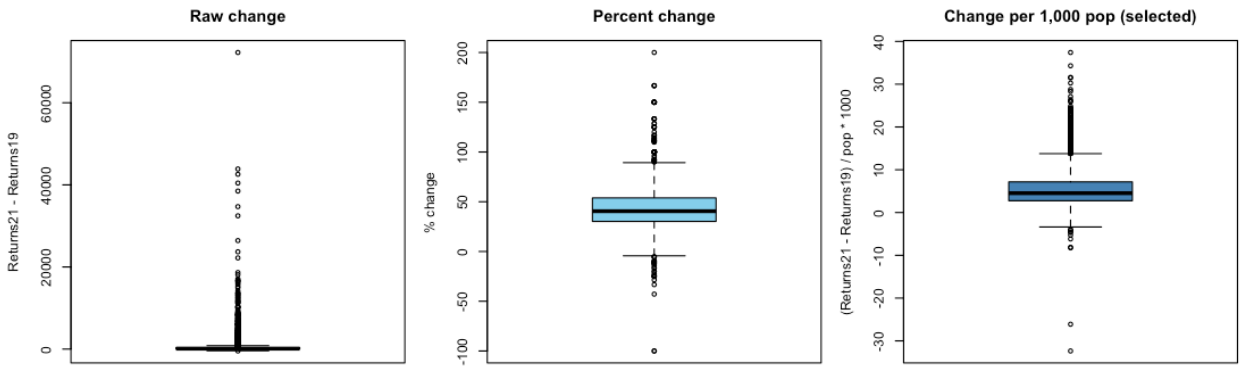


Figure 1: Distribution of the three candidate response variables. Raw change is dominated by large-county scale effects; percent change is unstable in small counties where Returns19 is small; change per 1,000 population is well-scaled and chosen as the response.

county-size dependence without amplifying small-denominator noise. It is also the natural unit for comparing growth rates across counties of different sizes.

```
resp_summ <- df %>%
  summarise(
    N = sum(!is.na(chg_returns_per_1k)),
    Min = min(chg_returns_per_1k, na.rm = TRUE),
    Q1 = quantile(chg_returns_per_1k, 0.25, na.rm = TRUE),
    Median = median(chg_returns_per_1k, na.rm = TRUE),
    Mean = mean(chg_returns_per_1k, na.rm = TRUE),
    Q3 = quantile(chg_returns_per_1k, 0.75, na.rm = TRUE),
    Max = max(chg_returns_per_1k, na.rm = TRUE),
    SD = sd(chg_returns_per_1k, na.rm = TRUE),
    PctPos = mean(chg_returns_per_1k > 0, na.rm = TRUE) * 100) %>%
  mutate(across(where(is.numeric), round, 2))

resp_summ
```

```
##      N    Min   Q1 Median Mean   Q3   Max   SD PctPos
## 1 3134 -32.36 2.76   4.52 5.52 7.16 37.41 4.44  95.12
```

The response is mostly positive: the median county added about 4 to 5 stub-8 filings per 1,000 residents between 2019 and 2021, and roughly 95 percent of counties post positive values. As noted in the introduction, this is consistent with bracket-creep and capital-gains realizations rather than zero-sum migration.

3.1 Box-Cox transformation

The response is right-skewed and contains a small number of negative values, both of which violate OLS homoskedasticity and normality assumptions. To fix this I apply a Box-Cox power transformation: shift the response to be strictly positive, then find the lambda that maximizes the Box-Cox log-likelihood and apply that power transformation. All regression models below use the Box-Cox-transformed response; the raw `chg_returns_per_1k` is retained for descriptive maps, boxplots, and the conclusion's summary statistics.

```r
# Keep the raw per-1k response for descriptive plots and reporting
df <- df %>% mutate(response_raw = chg_returns_per_1k)

# Shift to be strictly positive, then find the optimal Box-Cox lambda
shift_val <- abs(min(df$response_raw, na.rm = TRUE)) + 1
df <- df %>% mutate(response_shifted = response_raw + shift_val)

bc_result <- MASS::boxcox(response_shifted ~ 1, data = df,
                          lambda = seq(-2, 2, 0.1), plotit = FALSE)
lambda_opt <- bc_result$x[which.max(bc_result$y)]
cat("Box-Cox optimal lambda:", round(lambda_opt, 2), "\n")
```

```
## Box-Cox optimal lambda: 0.6
```

```r
df <- df %>% mutate(
  response_var = if (abs(lambda_opt) < 0.05) log(response_shifted)
                 else (response_shifted ^ lambda_opt - 1) / lambda_opt)
```

4 Issue with EITC?

The Earned Income Tax Credit is targeted at low and moderate income working households. Eligibility phases out well below $200,000 of AGI, so at stub 8 there are no EITC values by design. The four EITC-related columns in the dataset should therefore be zero everywhere in the stub-8 sample, and they are:

```r
eitc_cols <- c("pct_filers_eitc", "eitc_per_return",
               "eitc_per_claimant", "refund_eitc_proxy_per_return")
sapply(df[eitc_cols], function(v) {
  c(min = suppressWarnings(min(v, na.rm = TRUE)),
    max = suppressWarnings(max(v, na.rm = TRUE)),
    mean = suppressWarnings(mean(v, na.rm = TRUE)),
    n_nonzero = sum(v != 0, na.rm = TRUE),
    n_NA = sum(is.na(v)))})
```

```
##          pct_filers_eitc eitc_per_return eitc_per_claimant
## min                    0               0               Inf
## max                    0               0              -Inf
## mean                   0               0               NaN
## n_nonzero              0               0                 0
## n_NA                  50              50              3134
##          refund_eitc_proxy_per_return
## min                                 0
## max                                 0
## mean                                0
## n_nonzero                           0
## n_NA                               50
```

These are structural zeros rather than missing data. The same applies to the refundable child-tax-credit and refundable education-credit columns at this stub. Variables with zero variance break `regsubsets` (which requires non-constant predictors), so they must be excluded from the group.

```
zero_var_cols <- c("agi_stub", "pct_filers_eitc", "eitc_per_return",
                   "eitc_per_claimant", "refund_eitc_proxy_per_return",
                   "refund_edu_per_return")
```

5 Candidate Pool

The candidate pool is built by taking every numeric column and removing four kinds of variables: identifiers (FIPS codes, names), response variants (the change measures themselves and the raw 2019 to 2022 return counts that produced them), pre-summed totals (aggregate AGI, total tax liability, etc., which are mechanically tied to the response), and the structural zeros from the EITC check.

```
identifiers <- c("stname", "ctyname", "fips", "STATEFIPS", "COUNTYFIPS")
response_like <- c("Returns19", "Returns20", "Returns21", "Returns22",
                   "Individuals19", "Individuals20",
                   "Individuals21", "Individuals22",
                   "returns", "individuals",
                   "chg_returns_raw", "chg_individuals_raw",
                   "chg_returns_per_1k", "chg_individuals_per_1k",
                   "pct_chg_returns", "pct_chg_individuals",
                   "response_var")

totals_or_redundant <- c("agi_total", "income_total",
                         "labor_income", "business_income",
                         "investment_income", "rental_income",
                         "retirement_income", "transfer_like_income",
                         "non_labor_income",
                         "tax_before_credits_total", "credits_total",
                         "tax_after_credits_total",
                         "total_tax_liability", "total_tax_payments",
                         "avg_individuals_per_return")

# Variables that are themselves derived from filer-count differences (the
# same upstream quantity that produces the response). Including them would
# let the model "predict" the response with a re-scaled version of itself.
# These are excluded from the candidate pool for the regression and only
# kept around for descriptive maps.

calculation_vars <- c("migration_rate_filers", "migration_rate_persons",
                      "net_migration_filers", "net_migration_persons",
                      "inflow_filers", "outflow_filers", "inflow_agi",
                      "outflow_agi", "net_income_migration")

candidate_pool <- setdiff(names(df)[sapply(df, is.numeric)],
                          c(identifiers, response_like,
                            totals_or_redundant, zero_var_cols,
                            calculation_vars))

data_clean <- df %>% dplyr::select(response_var, all_of(candidate_pool)) %>%
  na.omit()
```

A note on the exclusion: The migration variables in this dataset (`migration_rate_filers`, `migration_rate_persons`, `net_migration_filers`, `net_migration_persons`, `inflow_filers`, `outflow_filers`, `inflow_agi`,

`outflow_agi`, `net_income_migration`) are all built from the same year-over-year IRS filer counts that produce the response. Letting them into the regression would amount to predicting the change in filers with a rescaled version of the change in filers, which inflates adj R-squared without adding economic content. They are kept in the file for descriptive county maps but are not used in the model selection or fit.

6 Best Subsets in Three Sets:

The candidate pool is too wide for a single exhaustive `regsubsets` call (cost grows exponentially with the number of predictors, and `leaps` itself caps out around 30 variables). I split the pool into three disjoint thematic groups and run an exhaustive search within each. The winners from each group are then pooled and re-competed in a final exhaustive pass.

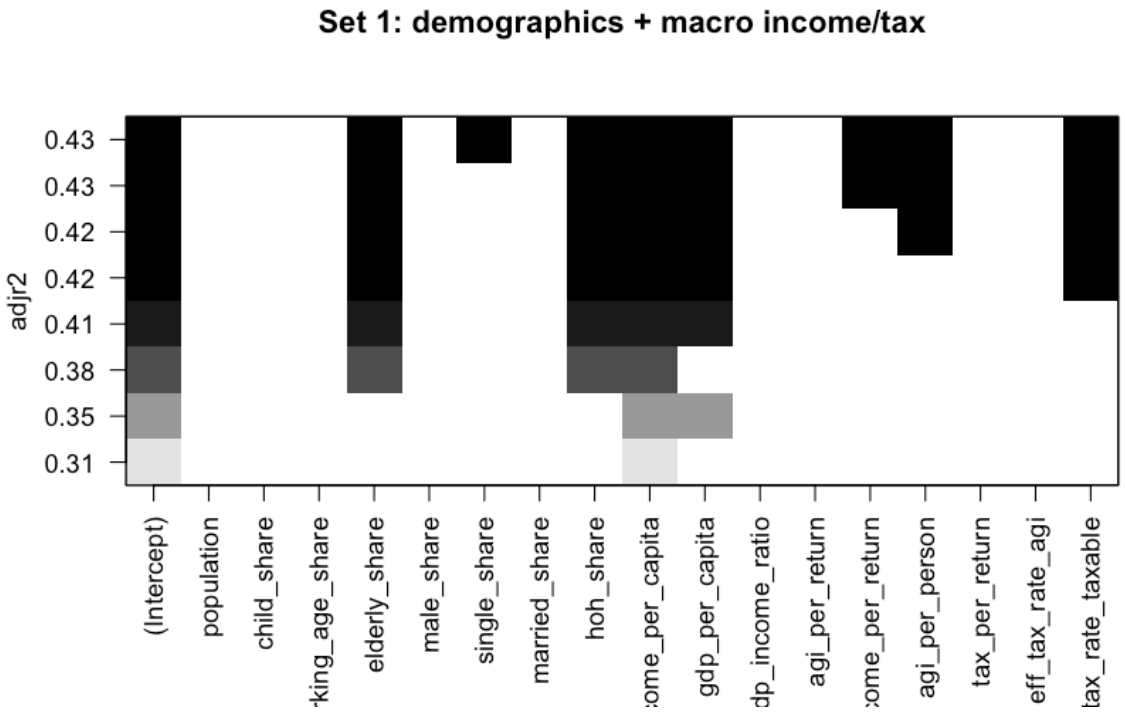
The three groups are: demographics and macro income/tax; income composition (where stub-8 income comes from); and tax structure, housing, and deductions. Migration variables are deliberately excluded from all three sets for the reason described above.

```
top_vars_from_subsets <- function(rs) {
  ss <- summary(rs)
  best_idx <- which.max(ss$adjr2)
  setdiff(names(coef(rs, best_idx)), "(Intercept)")
}

fit_and_diag <- function(rs, data, response = "response_var", title = "") {
  vars <- top_vars_from_subsets(rs)
  f   <- as.formula(paste(response, "~", paste(vars, collapse = " + ")))
  fit <- lm(f, data = data)
  cat("===", title, "===\n")
  cat("Best vars (", length(vars), "):\n  ",
      paste(vars, collapse = ", "), "\n", sep = "")
  cat("Adj R^2 = ", round(summary(fit)$adj.r.squared, 4),
      "  |  Resid SE = ", round(summary(fit)$sigma, 3), "\n", sep = "")
  par(mfrow = c(2, 2))
  plot(fit, main = title, cex = 0.4, col = adjustcolor("steelblue", 0.5))
  par(mfrow = c(1, 1))
  list(vars = vars, fit = fit, adjr2 = summary(fit)$adj.r.squared)
}
```

6.1 Set 1: Demographics and macro income/tax

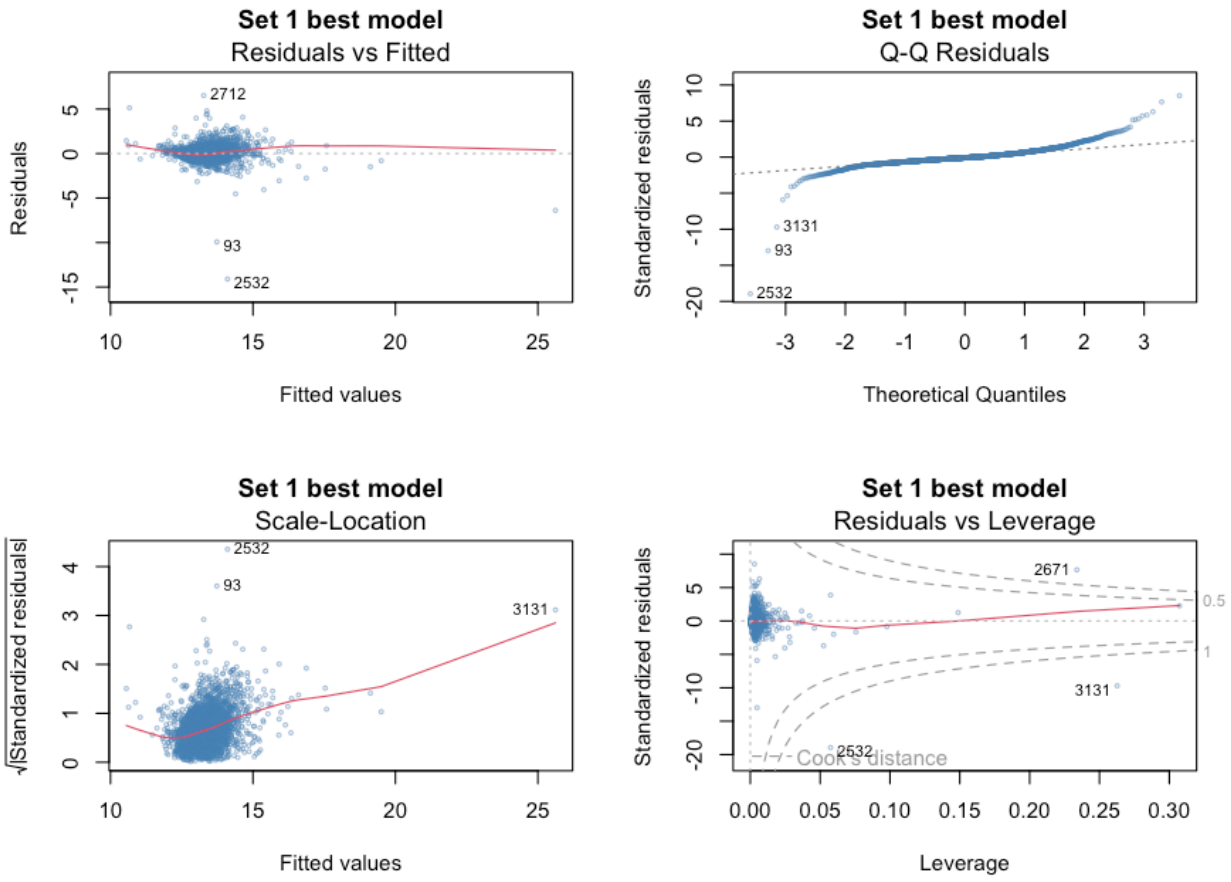
```
set1_vars <- intersect(c(
  "population", "child_share", "working_age_share",
  "elderly_share", "male_share",
  "single_share", "married_share", "hoh_share",
  "income_per_capita", "gdp_per_capita", "gdp_income_ratio",
  "agi_per_return", "income_per_return", "agi_per_person",
  "tax_per_return", "eff_tax_rate_agi", "eff_tax_rate_taxable"), candidate_pool)

rs1 <- regsubsets(response_var ~ .,
                  data = data_clean[, c("response_var", set1_vars)],
                  nvmax = 8, nbest = 1, method = "exhaustive")
```

```
plot(rs1, scale = "adjr2", main = "Set 1: demographics + macro income/tax")
```




```
set1 <- fit_and_diag(rs1, data_clean, title = "Set 1 best model")
```

```
## === Set 1 best model ===
## Best vars (8):
##   elderly_share, single_share, hoh_share, income_per_capita, gdp_per_capita, income_per_return, agi_
## Adj R^2 = 0.4291  |  Resid SE = 0.767
```

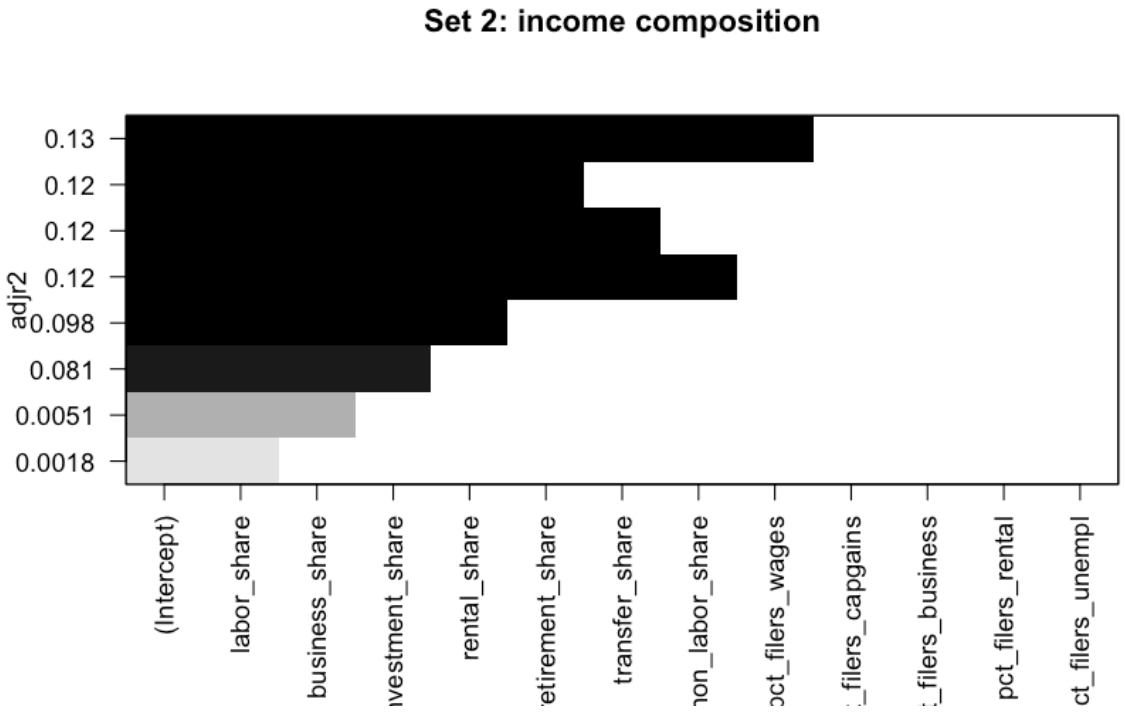


The diagnostic panel shows the heteroskedastic fan in residuals-vs-fitted that is typical when dollar-valued predictors enter on the linear scale, plus heavy tails in the QQ plot. Both will be addressed by the log transformations applied later. For now I keep all winners from this set without modification and move on.

6.2 Set 2: Income composition

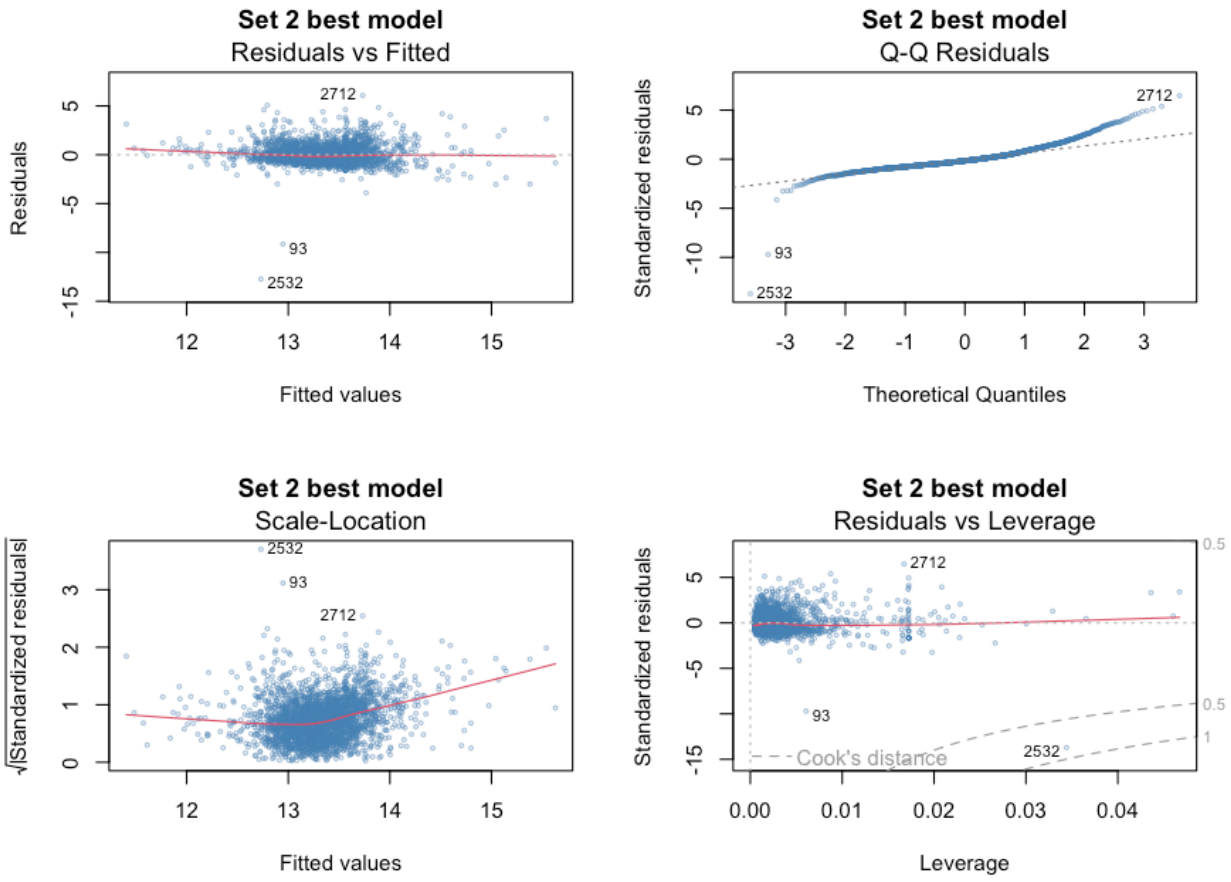
The income-share variables (`labor_share`, `business_share`, `investment_share`, and so on) sum to roughly one by construction, so `leaps` will report a small number of linear dependencies in this set. That is expected.

```
set2_vars <- intersect(c(
  "labor_share", "business_share", "investment_share",
  "rental_share", "retirement_share",
  "transfer_share", "non_labor_share",
  "pct_filers_wages", "pct_filers_capgains", "pct_filers_business",
  "pct_filers_rental", "pct_filers_unempl"), candidate_pool)

rs2 <- regsubsets(response_var ~ .,
                  data = data_clean[, c("response_var", set2_vars)],
                  nvmax = 8, nbest = 1, method = "exhaustive")

plot(rs2, scale = "adjr2", main = "Set 2: income composition")
```




```
set2 <- fit_and_diag(rs2, data_clean, title = "Set 2 best model")
```

```
## === Set 2 best model ===
## Best vars (8):
##   labor_share, business_share, investment_share, rental_share, retirement_share, transfer_share, non
## Adj R^2 = 0.1332  |  Resid SE = 0.945
```



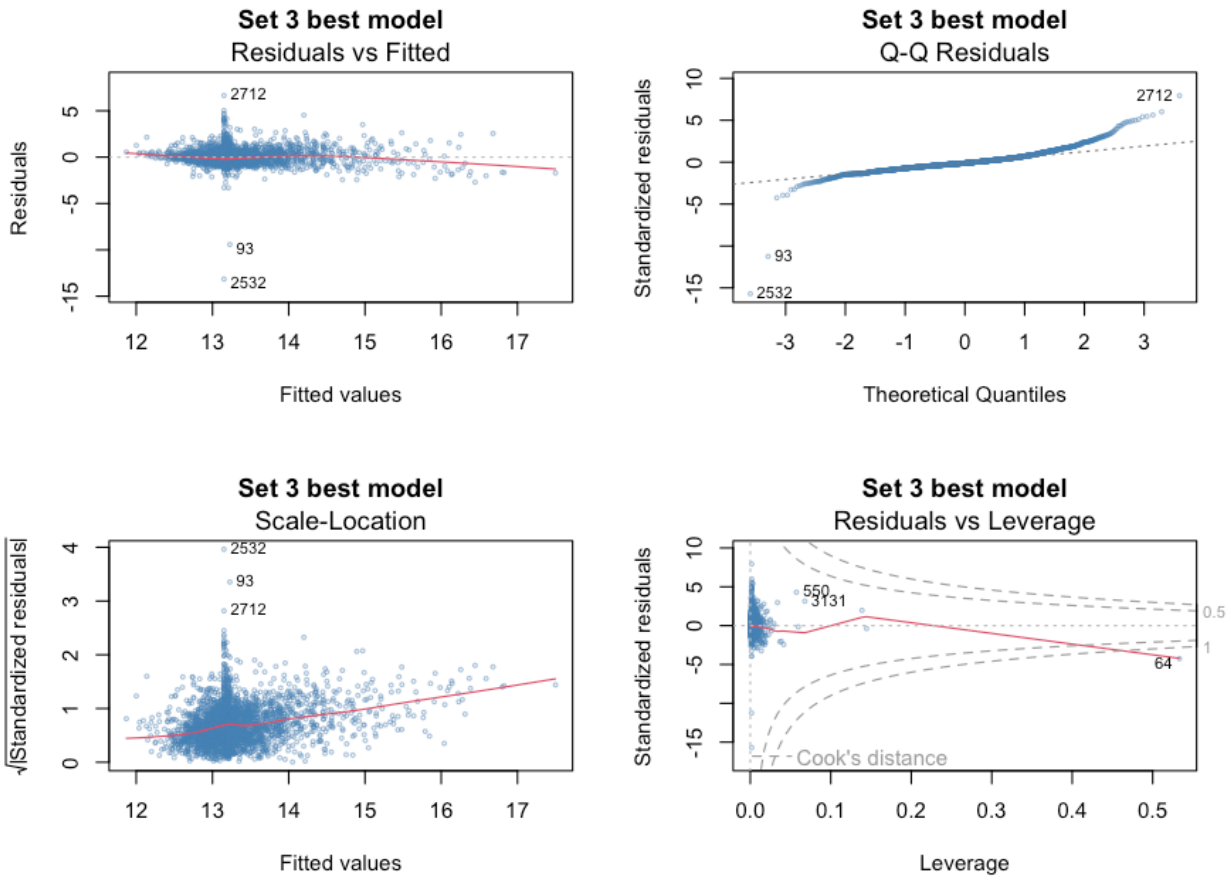
Income-share predictors are weaker in isolation than the Set 1 demographics. That makes sense: at the top bracket the income mix is fairly homogeneous across counties (mostly wages and capital gains), so there is less between-county variation for the model to use. The set still contributes its top winners to the pool.

6.3 Set 3: Tax structure, housing, and deductions

```
set3_vars <- intersect(c(
  "mortgage_per_return", "proptax_per_return",
  "salt_income_per_return", "salt_sales_per_return",
  "pct_filers_mortgage", "pct_filers_proptax",
  "credits_share",
  "refund_ctc_per_return"), candidate_pool)

rs3 <- regsubsets(response_var ~ .,
                  data = data_clean[, c("response_var", set3_vars)],
                  nvmax = 8, nbest = 1, method = "exhaustive")

plot(rs3, scale = "adjr2", main = "Set 3: tax structure / housing / deductions")
```


```
set3 <- fit_and_diag(rs3, data_clean, title = "Set 3 best model")
```

```
## === Set 3 best model ===
## Best vars (8):
##   mortgage_per_return, proptax_per_return, salt_income_per_return, salt_sales_per_return, pct_filers
## Adj R^2 = 0.3176  |  Resid SE = 0.838
```



This set isolates the housing-deduction and state-tax channel that Sets 1 and 2 cannot see. The mortgage interest and property tax deductions per return are the strongest predictors here; they essentially trace the geography of expensive housing markets (the coastal metros and a handful of mountain-resort counties) where the stub-8 population is densest and growing. The SALT income tax deduction adds a smaller but distinct signal coming from the high-state-tax states (NY, NJ, CA, MA, IL).

7 Pooling and Final Reduction

7.1 Pool the three winners

```
pooled_vars <- unique(c(set1$vars, set2$vars, set3$vars))
pooled_vars
```

```
##  [1] "elderly_share"          "single_share"           "hoh_share"
##  [4] "income_per_capita"      "gdp_per_capita"         "income_per_return"
##  [7] "agi_per_person"         "eff_tax_rate_taxable"   "labor_share"
## [10] "business_share"         "investment_share"       "rental_share"
## [13] "retirement_share"       "transfer_share"         "non_labor_share"
## [16] "pct_filers_wages"       "mortgage_per_return"    "proptax_per_return"
## [19] "salt_income_per_return" "salt_sales_per_return"  "pct_filers_mortgage"
## [22] "pct_filers_proptax"     "credits_share"          "refund_ctc_per_return"
```

7.2 Correlation

Before running the final best-subsets pass I check the correlation structure of the pooled candidates and drop near-duplicates.

```
pooled_cor <- cor(data_clean[, pooled_vars])
corrplot(pooled_cor, method = "color", type = "upper",
         tl.cex = 0.7, addCoef.col = "grey30", number.cex = 0.55)
```

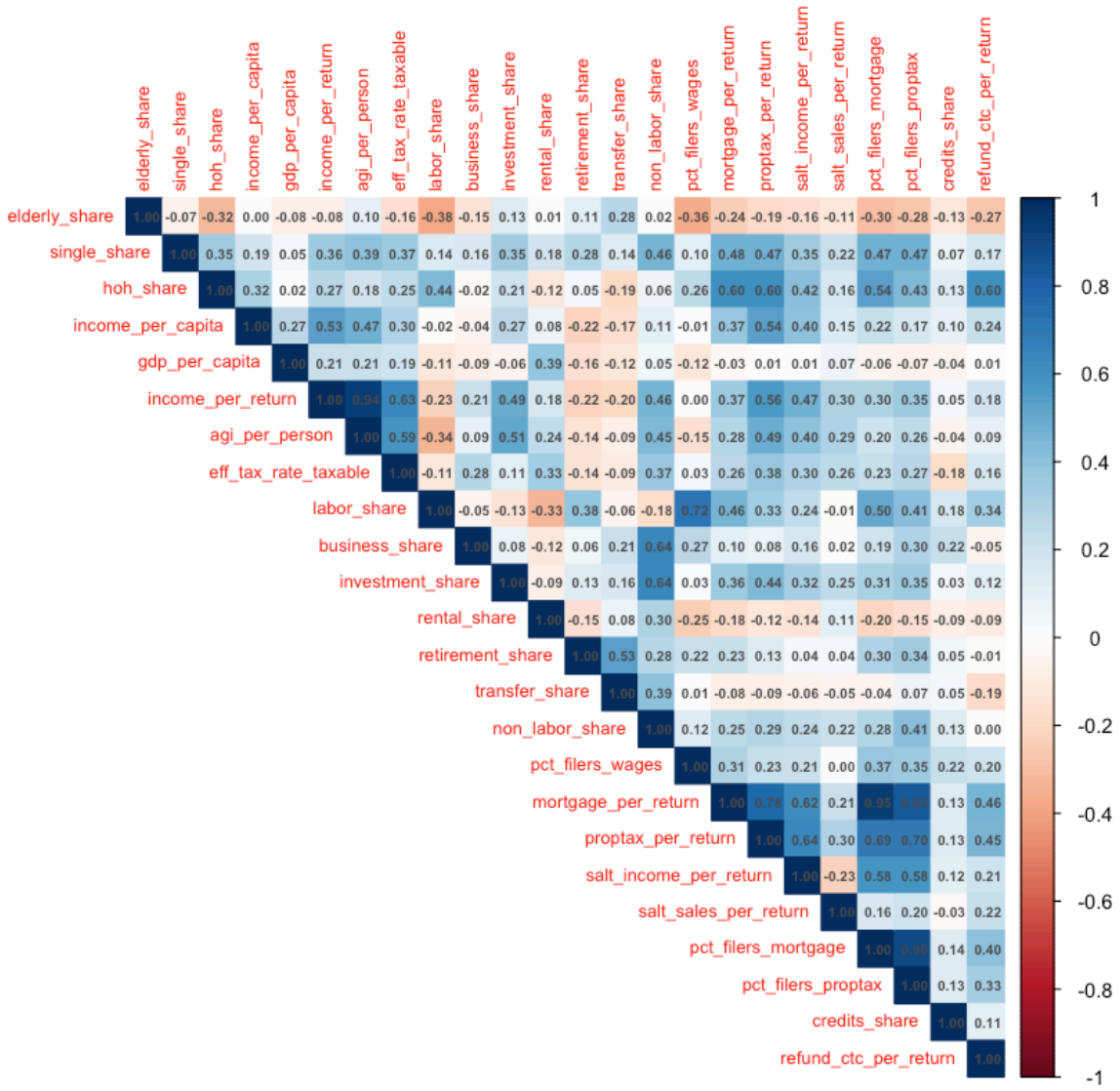


Figure 2: Correlation matrix of the pooled candidates from Sets 1 to 3. Pairs above $|r| = 0.85$ are dropped before the final best-subsets pass.

```
cor_threshold <- 0.85

hi_corr <- which(abs(pooled_cor) > cor_threshold & upper.tri(pooled_cor),
                 arr.ind = TRUE)

if (nrow(hi_corr) > 0) {
  to_drop <- unique(colnames(pooled_cor)[hi_corr[, "col"]])
  pairs_dropped <- apply(hi_corr, 1, function(ij) {
    sprintf("%s <-> %s (r=%.2f)",
            rownames(pooled_cor)[ij[1]], colnames(pooled_cor)[ij[2]],
            pooled_cor[ij[1], ij[2]])
  })
  cat("Pairs above |r|=", cor_threshold, ":\n  ", sep = "")
  cat(paste(pairs_dropped, collapse = "\n  "), "\n", sep = "")
  cat("Dropped:\n  ", paste(to_drop, collapse = ", "), "\n", sep = "")
  pooled_vars <- setdiff(pooled_vars, to_drop)
} else {
  cat("No high-correlation duplicates above ", cor_threshold, "\n", sep = "")
}
```

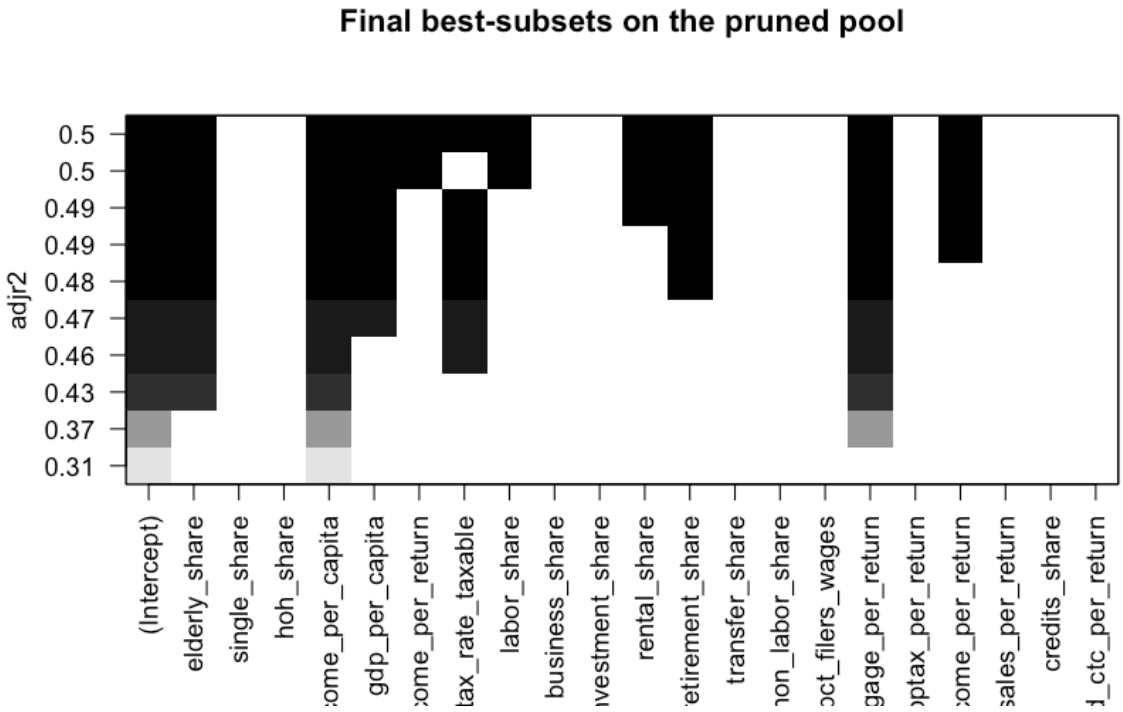
```
## Pairs above |r|=0.85:
##   income_per_return <-> agi_per_person (r=0.94)
##   mortgage_per_return <-> pct_filers_mortgage (r=0.95)
##   pct_filers_mortgage <-> pct_filers_proptax (r=0.90)
## Dropped:
##   agi_per_person, pct_filers_mortgage, pct_filers_proptax
```

```
cat("Pool after pruning:", length(pooled_vars), "vars\n")
```

```
## Pool after pruning: 21 vars
```

7.3 Final best-subsets pass

```
data_pooled <- data_clean[, c("response_var", pooled_vars)]
rs_final <- regsubsets(response_var ~ ., data = data_pooled,
                       nvmax = 10, nbest = 1, method = "exhaustive")
plot(rs_final, scale = "adjr2",
     main = "Final best-subsets on the pruned pool")
```




```
ss_final     <- summary(rs_final)
adjr2_size   <- round(ss_final$adjr2, 4)
cat("Adj R^2 by model size:\n"); print(adjr2_size)
```

```
## Adj R^2 by model size:
```

```
##  [1] 0.3096 0.3662 0.4339 0.4574 0.4699 0.4813 0.4899 0.4943 0.4975 0.5004
```

To pick the model size, I take the smallest model whose adj R-squared is within 0.005 of the maximum, capped at 8 predictors.

```
max_adj  <- max(ss_final$adjr2)
elbow <- min(which(ss_final$adjr2 >= max_adj - 0.005))
top_size <- min(elbow, 8)
top_vars <- setdiff(names(coef(rs_final, top_size)), "(Intercept)")

cat("Chosen size:", top_size, "  elbow=", elbow,
    "  argmax adj R^2 size=", which.max(ss_final$adjr2), "\n", sep = " ")
```

```
## Chosen size: 8   elbow= 9   argmax adj R^2 size= 10
```

```
cat("Top vars:\n  ", paste(top_vars, collapse = ", "), "\n", sep = "")
```

```
## Top vars:
##   elderly_share, income_per_capita, gdp_per_capita, eff_tax_rate_taxable, rental_share, retirement_sh
```

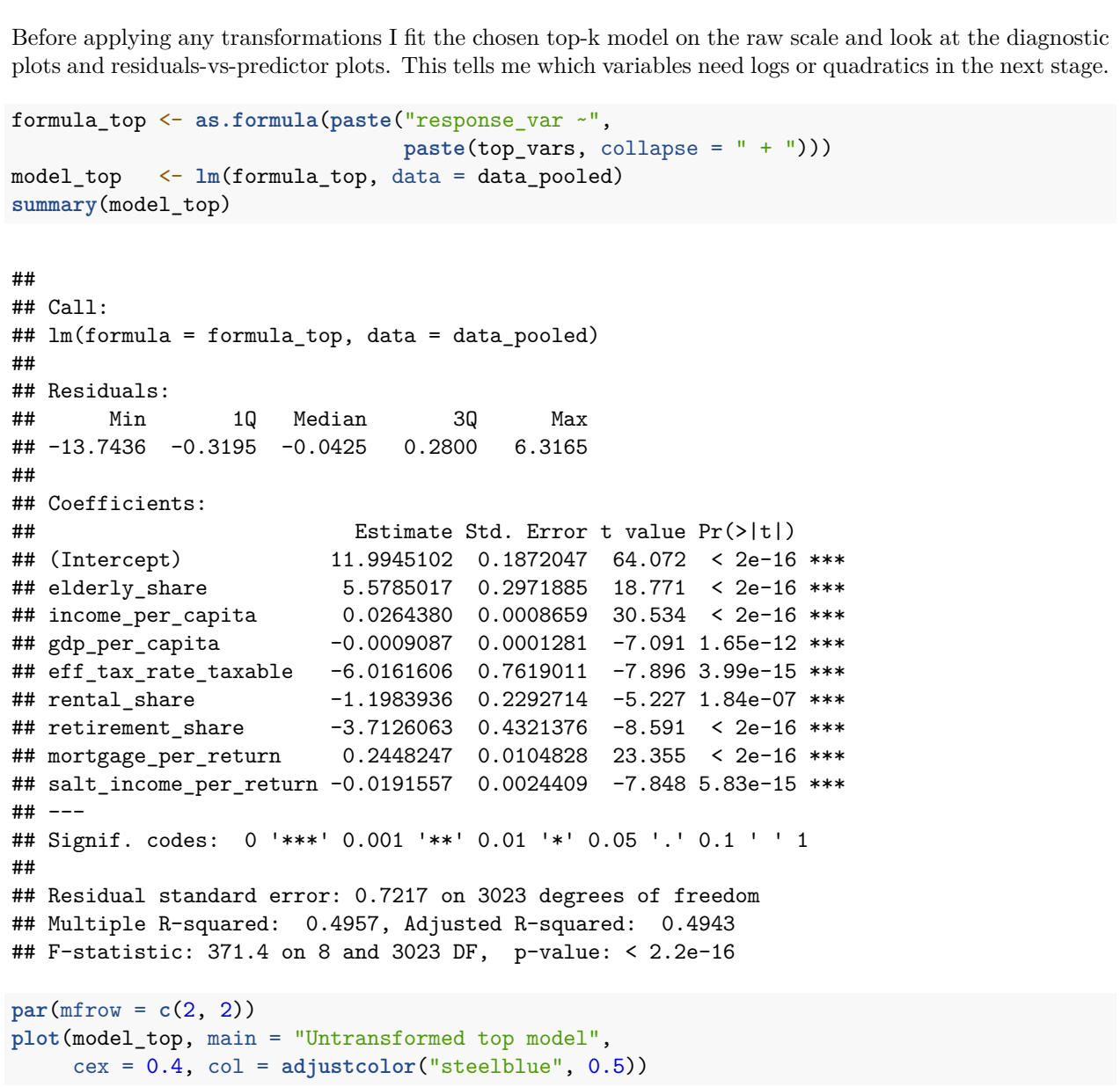
7.4 Untransformed top model + residuals

Before applying any transformations I fit the chosen top-k model on the raw scale and look at the diagnostic plots and residuals-vs-predictor plots. This tells me which variables need logs or quadratics in the next stage.

```
formula_top <- as.formula(paste("response_var ~",
                                paste(top_vars, collapse = " + ")))
model_top   <- lm(formula_top, data = data_pooled)
summary(model_top)
```

```
##
## Call:
## lm(formula = formula_top, data = data_pooled)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.7436  -0.3195  -0.0425   0.2800   6.3165
##
## Coefficients:
##                            Estimate Std. Error t value Pr(>|t|)
## (Intercept)            11.9945102  0.1872047  64.072  < 2e-16 ***
## elderly_share           5.5785017  0.2971885  18.771  < 2e-16 ***
## income_per_capita       0.0264380  0.0008659  30.534  < 2e-16 ***
## gdp_per_capita         -0.0009087  0.0001281  -7.091 1.65e-12 ***
## eff_tax_rate_taxable   -6.0161606  0.7619011  -7.896 3.99e-15 ***
## rental_share           -1.1983936  0.2292714  -5.227 1.84e-07 ***
## retirement_share       -3.7126063  0.4321376  -8.591  < 2e-16 ***
## mortgage_per_return     0.2448247  0.0104828  23.355  < 2e-16 ***
## salt_income_per_return -0.0191557  0.0024409  -7.848 5.83e-15 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7217 on 3023 degrees of freedom
## Multiple R-squared:  0.4957, Adjusted R-squared:  0.4943
## F-statistic: 371.4 on 8 and 3023 DF,  p-value: < 2.2e-16
```

```
par(mfrow = c(2, 2))
plot(model_top, main = "Untransformed top model",
     cex = 0.4, col = adjustcolor("steelblue", 0.5))
```



```
par(mfrow = c(1, 1))
```

```
md <- model.frame(model_top)
rv <- resid(model_top)
nc <- ceiling(sqrt(length(top_vars)))
par(mfrow = c(nc, nc))
for (v in top_vars) {
  plot(md[[v]], rv,
       main = paste("Residuals vs", v),
       xlab = v, ylab = "Residual",
       pch = 20, col = adjustcolor("steelblue", 0.4))
  abline(h = 0, col = "red")
```

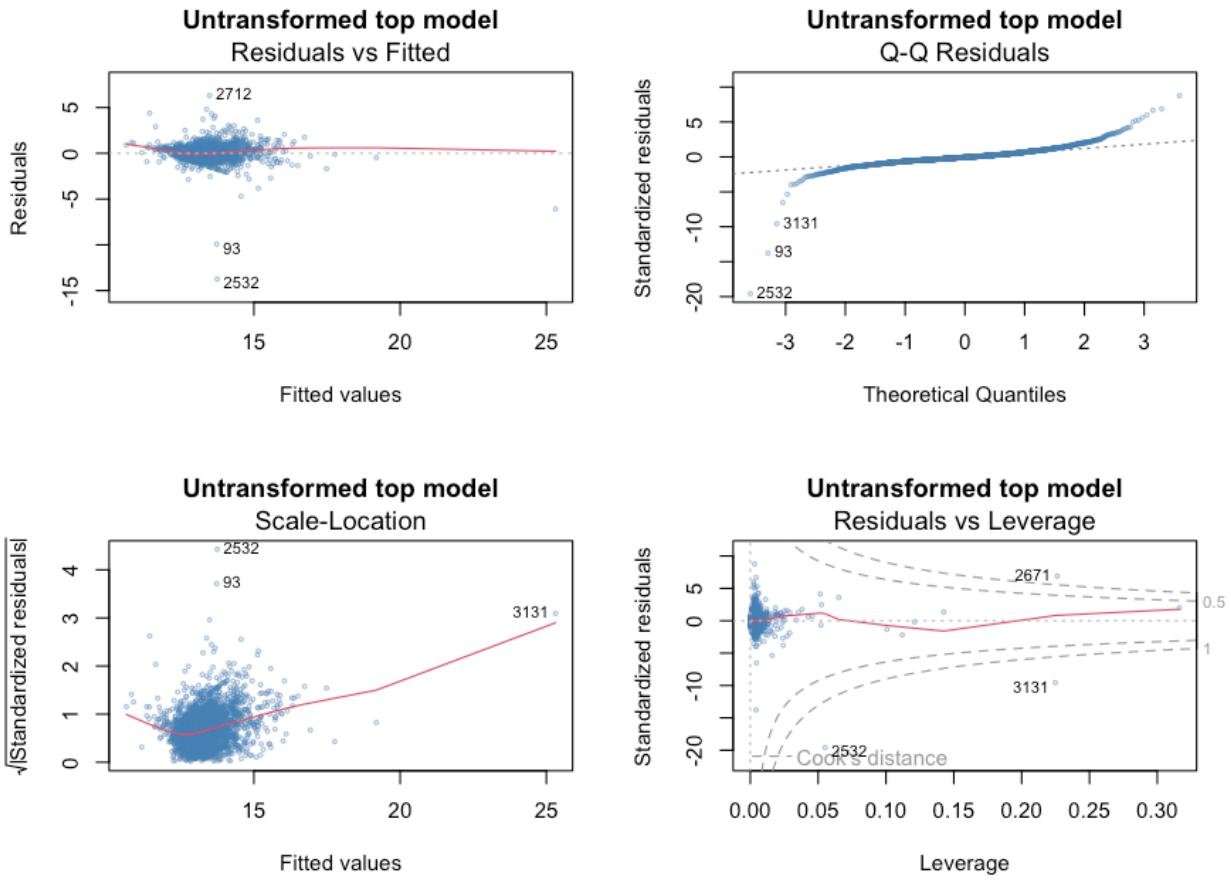


Figure 3: Standard diagnostic panels for the untransformed top model. The fan in residuals-vs-fitted and the heavy QQ tails motivate the log transformations applied in the next section.

```
}
par(mfrow = c(1, 1))
```

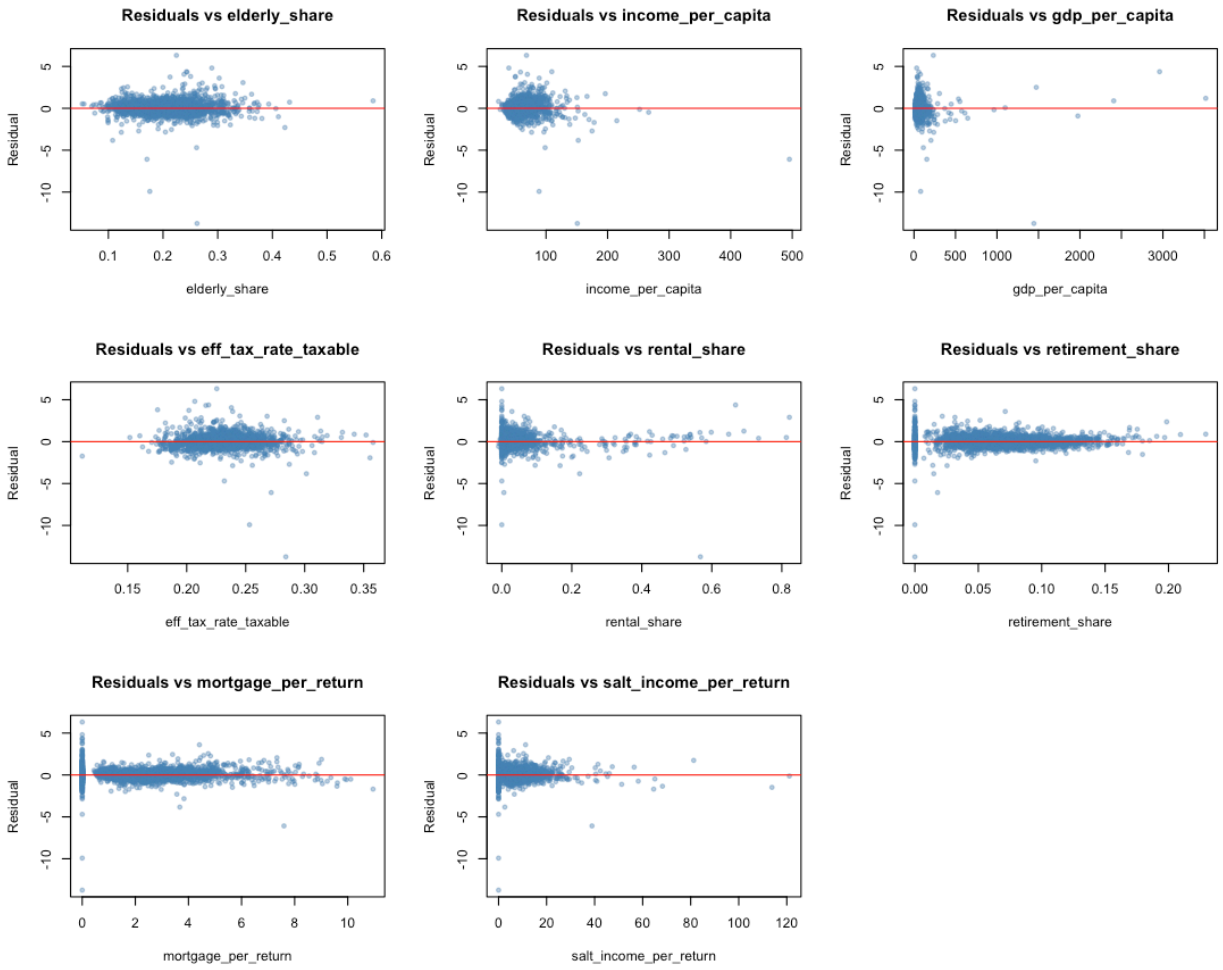


Figure 4: Residuals against each predictor in the untransformed top model. Predictors whose plots fan out or curve are logged (continuous dollar/count measures); pure shares are kept linear; the two age-share variables get an added quadratic term.

The fan in residuals-vs-fitted and the heavy QQ tails point to a log on the dollar- and count-valued predictors. The residuals-vs-predictor panels confirm it: every dollar measure shows a heavy right tail, while the pure proportion variables (shares) are well-behaved. The two age-share variables (`working_age_share`, `elderly_share`) sometimes show mild curvature, which motivates an added quadratic term.

7.5 Cook's distance trimming

Before refitting on the transformed scale, I trim observations that exert outsized influence on the OLS fit. These are typically resort and tourism counties (Aspen, Vail, Park City), university towns (Cambridge, Princeton), and energy-extraction counties (parts of North Dakota and west Texas) that behave qualitatively differently from the broader cross-section and disproportionately drag the fitted surface toward themselves. The rule used here is the standard threshold of Cook's distance greater than 4/n.

```r
cooksd <- cooks.distance(model_top)
n_obs <- nrow(data_pooled)
threshold <- 4 / n_obs
influential <- which(cooksd > threshold)

cat("Sample size before trim:", n_obs, "\n")
```

```
## Sample size before trim: 3032
```

```r
cat("Influential counties dropped (Cook's > 4/n):", length(influential), "\n")
```

```
## Influential counties dropped (Cook's > 4/n): 182
```

```r
cat("Sample size after trim:", n_obs - length(influential), "\n")
```

```
## Sample size after trim: 2850
```

```r
plot(cooksd, pch = 20, col = adjustcolor("steelblue", 0.5),
     main = "Cook's distance per county",
     xlab = "Observation index", ylab = "Cook's distance")
abline(h = threshold, col = "red", lty = 2)
```

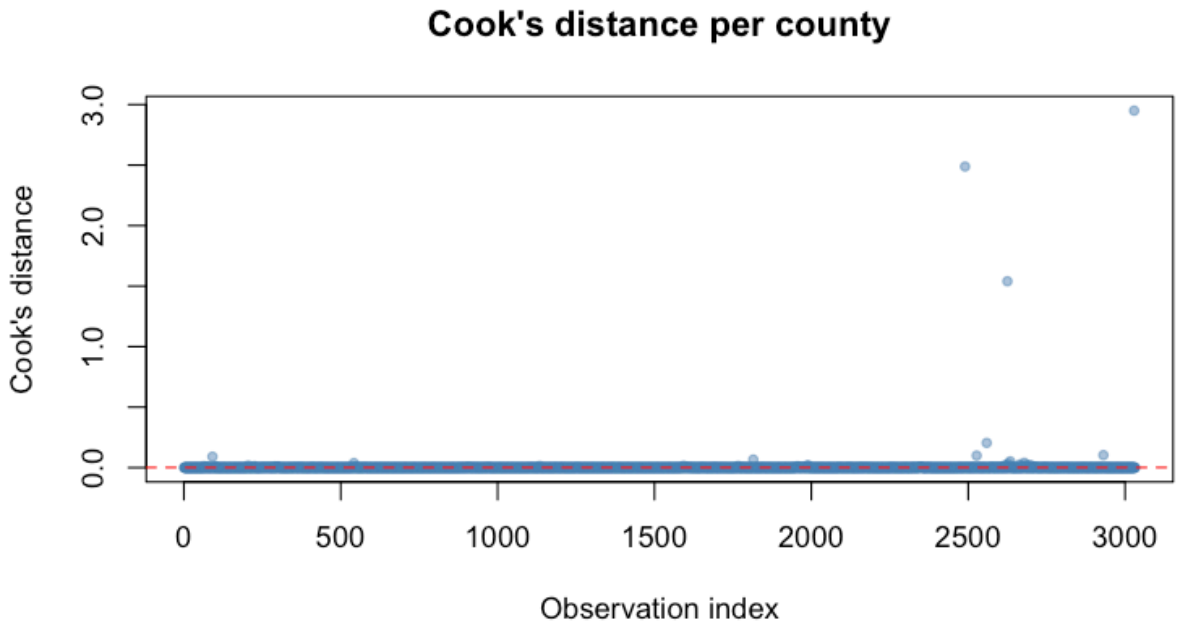


Figure 5: Cook's distance for each county. The horizontal line is the trimming threshold (4/n); counties above the line are removed from the regression sample before the transformed models are fit.

```
data_pooled_trim <- data_pooled[-influential, ]
data_clean_trim  <- data_clean[-influential, ]
```

The transformed models from Try-1 onwards are fit on `data_clean_trim`. The descriptive scatterplots in the next section are also drawn on the trimmed sample, since that is the sample the model actually estimates on.

8 Main Explanatory Variables

This section gives a one-paragraph description of each variable in the top-k selection, followed by a bivariate scatterplot against the response. The descriptions explain how each variable is constructed and the most likely economic reason it correlates with the change in stub-8 filings.

```
var_descriptions <- list(
  population = "Census Bureau county population estimate (2022). Larger counties host denser professiona
  income_per_capita = "BEA total personal income divided by Census population, in thousands of dollars p
  gdp_per_capita = "BEA county GDP divided by Census population. Closely related to income per capita bu
  agi_per_person = "Total county AGI (across all stubs) divided by population. A tax-side analogue of in
  agi_per_return = "Average AGI per stub-8 return in each county, in thousands of dollars. Captures with
  working_age_share = "Census share of population aged 25 to 64. The classic professional-hub indicator
  elderly_share = "Census share of population aged 65 or older. Two opposing mechanisms compete here. Re
  male_share = "Census share of male residents. A weaker predictor on its own. Partly proxies for indust
  married_share = "Census share of married households. Correlates with two-earner professional household
  labor_share = "Share of stub-8 AGI in the county that comes from wages and salaries (as opposed to cap
  retirement_share = "Share of stub-8 AGI from retirement sources (pensions, IRA withdrawals, etc.). Neg
  transfer_share = "Share of stub-8 AGI from transfer-like income (taxable Social Security, unemployment
  non_labor_share = "One minus labor share, so the complement: the share of stub-8 AGI from capital gain
  business_share = "Share of stub-8 AGI from sole-proprietor and pass-through business income. Highest i
  investment_share = "Share of stub-8 AGI from dividends, interest, and capital gains. Strongly co-moves
  rental_share = "Share of stub-8 AGI from rental real estate. Small at stub 8 nationally but concentrat
  tax_per_return = "Average federal income tax liability per stub-8 return in the county, in thousands o
  mortgage_per_return = "Average mortgage interest deduction per stub-8 return. High values are tightly
  proptax_per_return = "Average property tax deduction per stub-8 return. Geography mirrors mortgage int
  salt_income_per_return = "Average state-and-local income tax deduction per stub-8 return. Reflects exp
  salt_sales_per_return = "Average state-and-local sales tax deduction per stub-8 return. Filers can dec
  pct_filers_capgains = "Share of stub-8 returns in the county that report any capital gains income. A c
  pct_filers_mortgage = "Share of stub-8 returns that itemize mortgage interest. A complement to mortgag
  pct_filers_proptax = "Share of stub-8 returns that itemize property tax. Similar geographic pattern to
  credits_share = "Average share of pre-credit tax that ends up wiped out by federal tax credits (CTC, o
  refund_ctc_per_return = "Average refundable child tax credit received per stub-8 return. Essentially z
)
```

```
for (v in top_vars) {
  cat("##", v, "\n\n")
  desc <- var_descriptions[[v]]
  if (is.null(desc)) desc <- paste("Variable", v,
"selected by the best-subsets pipeline; no detailed description on file.")
  cat(desc, "\n\n")
}
```

8.1 elderly_share

Census share of population aged 65 or older. Two opposing mechanisms compete here. Retirement-heavy counties have less wage growth (a negative pull on the response) but realized very large capital gains during the 2020 to 2021 equity-market rally (a positive pull). The Florida coastal counties, Arizona retirement communities, and the South Carolina low-country show up clearly as high-elderly-share counties with fast stub-8 growth. Quadratic term captures the U-shape.

8.2 income_per_capita

BEA total personal income divided by Census population, in thousands of dollars per resident. The single strongest predictor in the bivariate scatter. The mechanism is threshold-crossing: counties whose income distribution already sits close to $200,000 see many filers cross the stub-8 line on even modest nominal income growth. A back-of-envelope intuition is that doubling income per capita roughly quadruples the number of households above the $200k threshold, because incomes are approximately log-normally distributed. Log-transformed in the model because the effect is multiplicative rather than linear.

8.3 gdp_per_capita

BEA county GDP divided by Census population. Closely related to income per capita but measures production rather than household earnings, so it picks up energy and manufacturing counties that have high output relative to wages (oil and gas counties in Texas and North Dakota, for example). Included as a robustness check on the income measure; log-transformed.

8.4 eff_tax_rate_taxable

Variable eff_tax_rate_taxable selected by the best-subsets pipeline; no detailed description on file.

8.5 rental_share

Share of stub-8 AGI from rental real estate. Small at stub 8 nationally but concentrated in resort and high-housing-cost counties.

8.6 retirement_share

Share of stub-8 AGI from retirement sources (pensions, IRA withdrawals, etc.). Negative or weak in the model: retirement income is more stable than wage or capital-gains income and contributed less to bracket creep over this window. Counties with high retirement_share tend to be retirement destinations where stub-8 growth is slower in absolute terms.

8.7 mortgage_per_return

Average mortgage interest deduction per stub-8 return. High values are tightly geographically concentrated in expensive coastal metros: the Bay Area, NYC suburbs, Boston, DC, San Diego, Seattle, Los Angeles. The variable is essentially a proxy for housing costs at the stub-8 level. Positively correlated with the response because expensive-housing counties have growing high-earner populations driven by both income growth and selective in-migration.

8.8 salt_income_per_return

Average state-and-local income tax deduction per stub-8 return. Reflects exposure to state-level income taxation; high in CA, NY, NJ, MA, OR, and a few other states. After the 2017 SALT cap of $10,000, this variable became much more relevant as a measure of net fiscal pressure on high earners.

The plot below shows one bivariate regression scatter per selected predictor: the predictor on the x-axis and the Box-Cox-transformed response on the y-axis. The red line is the OLS fit and the shaded band is its 95 percent confidence interval. The sample matches the trimmed sample used by the regression models (the high-Cook's counties are removed). None of these predictors are derived from the response, so each line is a genuine cross-county relationship.

```
scatter_df <- data_clean_trim %>%
  dplyr::select(response_var, all_of(top_vars)) %>%
  tidyr::pivot_longer(-response_var, names_to = "predictor", values_to = "x")

ggplot(scatter_df, aes(x = x, y = response_var)) +
  geom_point(alpha = 0.25, size = 0.9, color = "steelblue") +
  geom_smooth(method = "lm", color = "firebrick", fill = "salmon",
              linewidth = 0.8, alpha = 0.35) +
  facet_wrap(~ predictor, scales = "free_x", ncol = 3) +
  labs(x = NULL, y = "Response (Box-Cox transformed)") +
  theme_classic(base_size = 10) +
  theme(strip.background = element_blank(),
        strip.text = element_text(size = 9))
```

A few personal observations from the scatter panel. Income per capita and AGI per person produce the cleanest linear relationships with the response; their confidence intervals are tight and the slope is unambiguous. The two age-share variables (working-age and elderly) both show visible curvature, which is exactly what motivates the quadratic terms applied in the next section. The deduction variables (mortgage per return, property tax per return, SALT income per return) are right-skewed with a long tail of expensive-coastal-metro counties; in the linear scale this looks like a dense cluster near zero with a few outliers far to the right, which is why these variables are logged in the next step. The pure share variables (labor share, retirement share) are well-behaved and stay on the linear scale.

9 Transformations and Interactions

The diagnostic plots above motivate three changes: log the heavy-tailed predictors, add quadratic terms for the two age shares, and let the predictors interact pairwise (since the threshold-crossing mechanism depends on combinations like income x population, not on either variable alone).

9.1 Try-1: log + quadratic main effects

```
log_candidates <- c("population", "income_per_capita", "gdp_per_capita",
                    "tax_per_return", "mortgage_per_return",
                    "proptax_per_return",
                    "salt_income_per_return", "salt_sales_per_return",
                    "agi_per_return", "income_per_return",
                    "agi_per_person",
                    "net_income_migration", "inflow_agi", "outflow_agi",
```

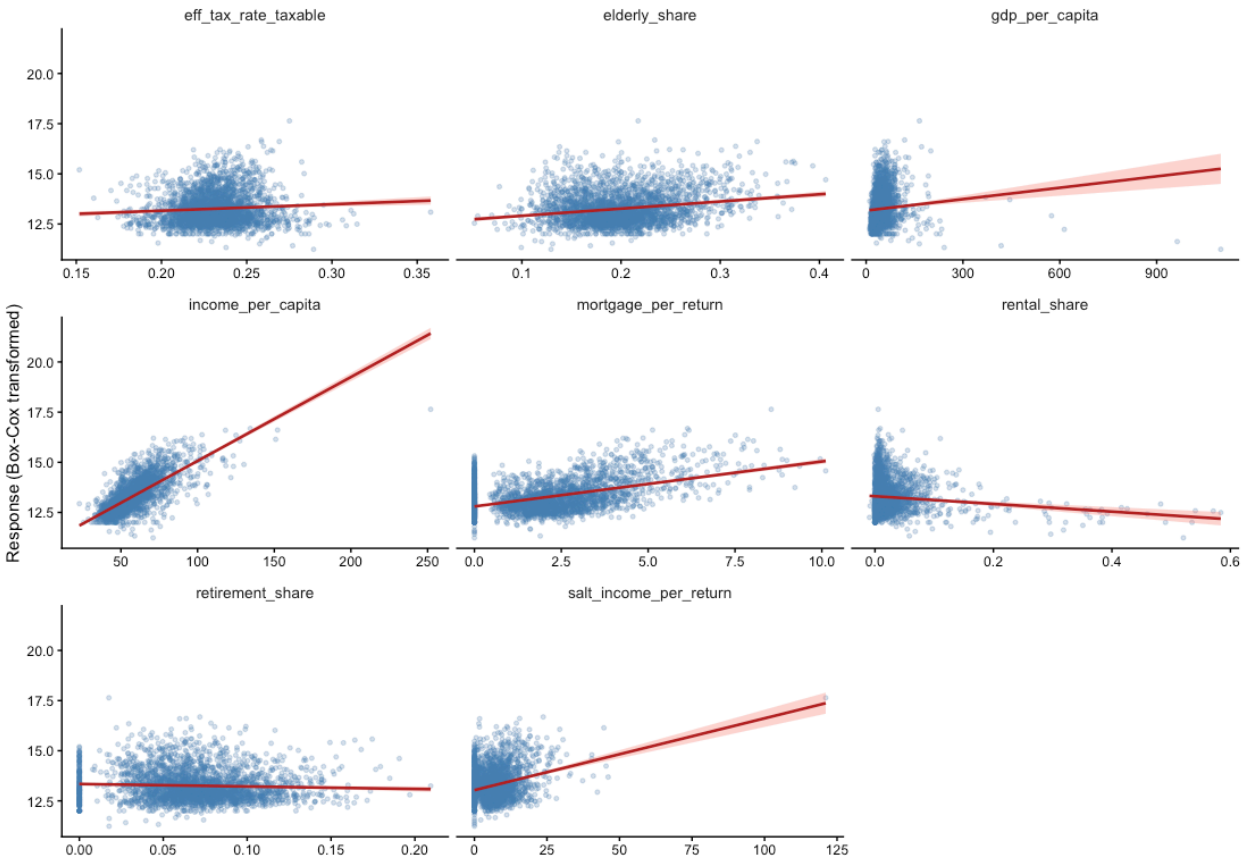


Figure 6: Bivariate regression scatterplots of each selected predictor against the Box-Cox-transformed response. The red line is the OLS fit; the shaded band is its 95 percent confidence interval. Predictors that curve or fan are logged or quadratically transformed in the next section.

```
                    "inflow_filers", "outflow_filers",
                    "refund_ctc_per_return")
needs_log  <- intersect(top_vars, log_candidates)
quad_terms <- intersect(top_vars,
                        c("working_age_share", "elderly_share"))

# Build the transformed dataset on the Cook's-trimmed sample so all
# Try-1 through Try-4 models are estimated on the same observations.
data_transformed <- data_clean_trim
for (v in needs_log) {
  data_transformed[[paste0("log_", v)]] <-
    log(pmax(data_transformed[[v]], 1e-6))
}

base_rhs <- sapply(top_vars, function(v) {
  if (v %in% needs_log) paste0("log_", v) else v
})
if (length(quad_terms) > 0) {
  base_rhs <- c(base_rhs, paste0("I(", quad_terms, "^2)"))
}
base_terms_returns <- unname(base_rhs)

cat("Logged predictors:\n  ", paste(needs_log,  collapse = ", "), "\n", sep = "")
```

```
## Logged predictors:
##   income_per_capita, gdp_per_capita, mortgage_per_return, salt_income_per_return
```

```
cat("Quadratic terms:\n  ", paste(quad_terms, collapse = ", "), "\n", sep = "")
```

```
## Quadratic terms:
##   elderly_share
```

```
cat("Try-1 right-hand side:\n  ",
    paste(base_terms_returns, collapse = ", "), "\n", sep = "")
```

```
## Try-1 right-hand side:
##   elderly_share, log_income_per_capita, log_gdp_per_capita, eff_tax_rate_taxable, rental_share, reti
```

```
model_try1 <- lm(as.formula(paste("response_var ~",
                                  paste(base_terms_returns, collapse = " + "))),
                 data = data_transformed)
summary(model_try1)
```

```
##
## Call:
## lm(formula = as.formula(paste("response_var ~", paste(base_terms_returns,
##     collapse = " + "))), data = data_transformed)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.76056 -0.28516 -0.03733  0.25386  2.15553
```

```
##
## Coefficients:
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                  4.056149   0.244892  16.563  < 2e-16 ***
## elderly_share               -9.323453   1.164072  -8.009 1.67e-15 ***
## log_income_per_capita        2.955957   0.054003  54.737  < 2e-16 ***
## log_gdp_per_capita          -0.257805   0.026013  -9.911  < 2e-16 ***
## eff_tax_rate_taxable        -3.835599   0.536718  -7.146 1.13e-12 ***
## rental_share                -1.708268   0.200157  -8.535  < 2e-16 ***
## retirement_share            -1.313775   0.324354  -4.050 5.25e-05 ***
## log_mortgage_per_return      0.014860   0.001963   7.568 5.07e-14 ***
## log_salt_income_per_return   0.002939   0.001806   1.627    0.104
## I(elderly_share^2)          29.370620   2.703579  10.864  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4656 on 2840 degrees of freedom
## Multiple R-squared:  0.6542, Adjusted R-squared:  0.6531
## F-statistic: 597.1 on 9 and 2840 DF,  p-value: < 2.2e-16
```

```
summary(model_try1)$adj.r.squared - summary(model_top)$adj.r.squared
```

```
## [1] 0.1588057
```

9.2 Try-2: full pairwise interactions

The base specification is expanded to include every pairwise product of the main-effect terms (quadratic terms are not interacted, only added back at the end).

```
main_effects <- base_terms_returns[!grepl("^I\\(", base_terms_returns)]
inter_rhs <- paste0("(", paste(main_effects, collapse = " + "), ")^2")
quad_rhs <- base_terms_returns[grepl("^I\\(", base_terms_returns)]
all_rhs <- if (length(quad_rhs) > 0)
  paste(inter_rhs, "+", paste(quad_rhs, collapse = " + ")) else inter_rhs

model_interaction <- lm(as.formula(paste("response_var ~", all_rhs)),
                        data = data_transformed)
summary(model_interaction)
```

```
##
## Call:
## lm(formula = as.formula(paste("response_var ~", all_rhs)), data = data_transformed)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.64560 -0.25692 -0.03476  0.22922  1.79195
##
## Coefficients:
##                                                    Estimate Std. Error
## (Intercept)                                      -2.211e+00  2.501e+00
## elderly_share                                    -3.277e+01  4.114e+00
## log_income_per_capita                             4.879e+00  7.504e-01
```

```
## log_gdp_per_capita                                    1.601e-01  4.669e-01
## eff_tax_rate_taxable                                  2.349e+01  9.608e+00
## rental_share                                         -4.830e+00  4.882e+00
## retirement_share                                      5.196e+00  7.029e+00
## log_mortgage_per_return                              -2.836e-01  4.706e-02
## log_salt_income_per_return                           -1.250e-01  4.310e-02
## I(elderly_share^2)                                    2.736e+01  3.260e+00
## elderly_share:log_income_per_capita                   3.271e+00  1.122e+00
## elderly_share:log_gdp_per_capita                      1.426e+00  5.320e-01
## elderly_share:eff_tax_rate_taxable                    1.920e+01  1.185e+01
## elderly_share:rental_share                            6.709e+00  4.907e+00
## elderly_share:retirement_share                        1.294e+01  6.609e+00
## elderly_share:log_mortgage_per_return                -4.339e-02  5.246e-02
## elderly_share:log_salt_income_per_return             -1.273e-01  4.707e-02
## log_income_per_capita:log_gdp_per_capita             -5.763e-02  8.181e-02
## log_income_per_capita:eff_tax_rate_taxable           -7.082e+00  2.945e+00
## log_income_per_capita:rental_share                   -1.012e+00  1.157e+00
## log_income_per_capita:retirement_share               -6.133e+00  1.938e+00
## log_income_per_capita:log_mortgage_per_return         8.992e-02  1.273e-02
## log_income_per_capita:log_salt_income_per_return      4.269e-02  1.137e-02
## log_gdp_per_capita:eff_tax_rate_taxable              -2.041e+00  1.337e+00
## log_gdp_per_capita:rental_share                      -1.462e-01  3.544e-01
## log_gdp_per_capita:retirement_share                   7.638e-01  7.901e-01
## log_gdp_per_capita:log_mortgage_per_return           -1.626e-02  5.216e-03
## log_gdp_per_capita:log_salt_income_per_return         8.628e-03  4.377e-03
## eff_tax_rate_taxable:rental_share                     2.308e+01  8.600e+00
## eff_tax_rate_taxable:retirement_share                 5.857e+01  1.765e+01
## eff_tax_rate_taxable:log_mortgage_per_return          1.752e-02  1.030e-01
## eff_tax_rate_taxable:log_salt_income_per_return      -1.806e-01  9.523e-02
## rental_share:retirement_share                         5.574e+00  1.060e+01
## rental_share:log_mortgage_per_return                 -8.314e-02  3.481e-02
## rental_share:log_salt_income_per_return              -3.577e-02  3.060e-02
## retirement_share:log_mortgage_per_return              1.308e-01  6.211e-02
## retirement_share:log_salt_income_per_return          -3.103e-02  5.768e-02
## log_mortgage_per_return:log_salt_income_per_return    4.555e-04  2.781e-04
##                                                      t value Pr(>|t|)
## (Intercept)                                           -0.884 0.376719
## elderly_share                                         -7.966 2.36e-15 ***
## log_income_per_capita                                  6.502 9.33e-11 ***
## log_gdp_per_capita                                     0.343 0.731662
## eff_tax_rate_taxable                                   2.445 0.014555 *
## rental_share                                          -0.989 0.322530
## retirement_share                                       0.739 0.459772
## log_mortgage_per_return                               -6.026 1.90e-09 ***
## log_salt_income_per_return                            -2.901 0.003751 **
## I(elderly_share^2)                                     8.394  < 2e-16 ***
## elderly_share:log_income_per_capita                    2.917 0.003563 **
## elderly_share:log_gdp_per_capita                       2.680 0.007399 **
## elderly_share:eff_tax_rate_taxable                     1.620 0.105377
## elderly_share:rental_share                             1.367 0.171685
## elderly_share:retirement_share                         1.958 0.050309 .
## elderly_share:log_mortgage_per_return                 -0.827 0.408244
## elderly_share:log_salt_income_per_return              -2.705 0.006868 **
## log_income_per_capita:log_gdp_per_capita              -0.704 0.481256
```

```
## log_income_per_capita:eff_tax_rate_taxable          -2.405 0.016240 *
## log_income_per_capita:rental_share                  -0.875 0.381539
## log_income_per_capita:retirement_share              -3.164 0.001572 **
## log_income_per_capita:log_mortgage_per_return        7.066 2.00e-12 ***
## log_income_per_capita:log_salt_income_per_return     3.756 0.000176 ***
## log_gdp_per_capita:eff_tax_rate_taxable             -1.527 0.126869
## log_gdp_per_capita:rental_share                     -0.412 0.680007
## log_gdp_per_capita:retirement_share                  0.967 0.333742
## log_gdp_per_capita:log_mortgage_per_return          -3.117 0.001843 **
## log_gdp_per_capita:log_salt_income_per_return        1.972 0.048764 *
## eff_tax_rate_taxable:rental_share                    2.684 0.007309 **
## eff_tax_rate_taxable:retirement_share                3.319 0.000916 ***
## eff_tax_rate_taxable:log_mortgage_per_return         0.170 0.864980
## eff_tax_rate_taxable:log_salt_income_per_return     -1.896 0.058046 .
## rental_share:retirement_share                        0.526 0.598892
## rental_share:log_mortgage_per_return                -2.388 0.017000 *
## rental_share:log_salt_income_per_return             -1.169 0.242544
## retirement_share:log_mortgage_per_return             2.105 0.035363 *
## retirement_share:log_salt_income_per_return         -0.538 0.590614
## log_mortgage_per_return:log_salt_income_per_return   1.638 0.101633
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.436 on 2812 degrees of freedom
## Multiple R-squared:  0.6998, Adjusted R-squared:  0.6958
## F-statistic: 177.1 on 37 and 2812 DF,  p-value: < 2.2e-16
```

```
cat("Try-2 adj R^2:", round(summary(model_interaction)$adj.r.squared, 4), "\n")
```

```
## Try-2 adj R^2: 0.6958
```

9.3 Try-3: keep only interactions significant at p < 0.05

```
extract_sig_interactions <- function(fit, alpha = 0.05) {
  ct <- summary(fit)$coefficients
  if (!"Pr(>|t|)" %in% colnames(ct)) return(character(0))
  nm <- rownames(ct)
  is_int <- grepl(":", nm, fixed = TRUE)
  pvals  <- ct[, "Pr(>|t|)"]
  keep   <- is_int & !is.na(pvals) & pvals < alpha
  nm[keep]
}

sig_int_returns <- extract_sig_interactions(model_interaction, alpha = 0.05)
cat("Significant interactions from Try-2 (", length(sig_int_returns), "):\n  ",
    paste(sig_int_returns, collapse = ", "), "\n", sep = "")
```

```
## Significant interactions from Try-2 (13):
##   elderly_share:log_income_per_capita, elderly_share:log_gdp_per_capita, elderly_share:log_salt_incon
```

```r
rhs_returns <- paste(c(base_terms_returns, sig_int_returns), collapse = " + ")
model_final <- lm(as.formula(paste("response_var ~", rhs_returns)),
                  data = data_transformed)
cat("Try-3 adj R^2:", round(summary(model_final)$adj.r.squared, 4), "\n")
```

```
## Try-3 adj R^2: 0.6938
```

```r
summary(model_final)
```

```
##
## Call:
## lm(formula = as.formula(paste("response_var ~", rhs_returns)),
##     data = data_transformed)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.70285 -0.25721 -0.03481  0.23413  1.82334
##
## Coefficients:
##                                                  Estimate Std. Error t value
## (Intercept)                                     -2.631897   2.177494  -1.209
## elderly_share                                  -29.106340   3.377164  -8.619
## log_income_per_capita                            5.172172   0.545563   9.480
## log_gdp_per_capita                              -0.607053   0.096599  -6.284
## eff_tax_rate_taxable                            31.250295   8.533791   3.662
## rental_share                                    -5.530889   1.991578  -2.777
## retirement_share                                 2.256762   6.148941   0.367
## log_mortgage_per_return                         -0.278905   0.038394  -7.264
## log_salt_income_per_return                      -0.165959   0.033431  -4.964
## I(elderly_share^2)                              30.690175   2.847389  10.778
## elderly_share:log_income_per_capita              2.876996   1.034457   2.781
## elderly_share:log_gdp_per_capita                 1.993508   0.494786   4.029
## elderly_share:log_salt_income_per_return        -0.131961   0.031062  -4.248
## log_income_per_capita:eff_tax_rate_taxable      -9.691181   2.113054  -4.586
## log_income_per_capita:retirement_share          -3.758690   1.418696  -2.649
## log_income_per_capita:log_mortgage_per_return    0.086846   0.011271   7.705
## log_income_per_capita:log_salt_income_per_return 0.040770   0.010438   3.906
## log_gdp_per_capita:log_mortgage_per_return      -0.014917   0.004354  -3.426
## log_gdp_per_capita:log_salt_income_per_return    0.009079   0.003988   2.276
## eff_tax_rate_taxable:rental_share               14.024136   7.500623   1.870
## eff_tax_rate_taxable:retirement_share           55.046374  14.280726   3.855
## rental_share:log_mortgage_per_return            -0.099867   0.026956  -3.705
## retirement_share:log_mortgage_per_return         0.094561   0.043874   2.155
##                                                 Pr(>|t|)
## (Intercept)                                     0.226886
## elderly_share                                    < 2e-16 ***
## log_income_per_capita                            < 2e-16 ***
## log_gdp_per_capita                              3.80e-10 ***
## eff_tax_rate_taxable                            0.000255 ***
## rental_share                                    0.005520 **
## retirement_share                                0.713634
## log_mortgage_per_return                         4.83e-13 ***
```

```
## log_salt_income_per_return                          7.31e-07 ***
## I(elderly_share^2)                                   < 2e-16 ***
## elderly_share:log_income_per_capita                 0.005452 **
## elderly_share:log_gdp_per_capita                    5.75e-05 ***
## elderly_share:log_salt_income_per_return            2.22e-05 ***
## log_income_per_capita:eff_tax_rate_taxable          4.71e-06 ***
## log_income_per_capita:retirement_share              0.008108 **
## log_income_per_capita:log_mortgage_per_return       1.79e-14 ***
## log_income_per_capita:log_salt_income_per_return    9.60e-05 ***
## log_gdp_per_capita:log_mortgage_per_return          0.000621 ***
## log_gdp_per_capita:log_salt_income_per_return       0.022894 *
## eff_tax_rate_taxable:rental_share                   0.061625 .
## eff_tax_rate_taxable:retirement_share               0.000119 ***
## rental_share:log_mortgage_per_return                0.000216 ***
## retirement_share:log_mortgage_per_return            0.031223 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4375 on 2827 degrees of freedom
## Multiple R-squared:  0.6962, Adjusted R-squared:  0.6938
## F-statistic: 294.4 on 22 and 2827 DF,  p-value: < 2.2e-16
```

9.4 Try-4: final best subsets on the Try-3 feature set

Try-3 carries every base term plus every significant interaction. To find the genuinely best-fitting final specification, I run one more exhaustive best-subsets pass over that full feature set and pick the size that maximizes adjusted R-squared. This is the same idea as the earlier set-level best-subsets sweeps, just applied at the feature-and-interaction level rather than the raw-variable level. Try-4 is the final model carried into the diagnostics and the side-by-side comparison.

```
mm <- model.matrix(model_final)[, -1]

clean_names <- function(x) {
  x <- gsub(":", "_x_", x, fixed = TRUE)
  x <- gsub("I\\(", "sq_", x)
  x <- gsub("\\^2\\)", "", x)
  x <- gsub("[()]", "", x)
  x
}
orig_names <- colnames(mm)
colnames(mm) <- clean_names(orig_names)
name_map <- setNames(orig_names, colnames(mm))

bss_df <- as.data.frame(mm)
bss_df$response_var <- data_transformed$response_var

cat("Design matrix:", nrow(mm), "rows x", ncol(mm), "features\n")
```

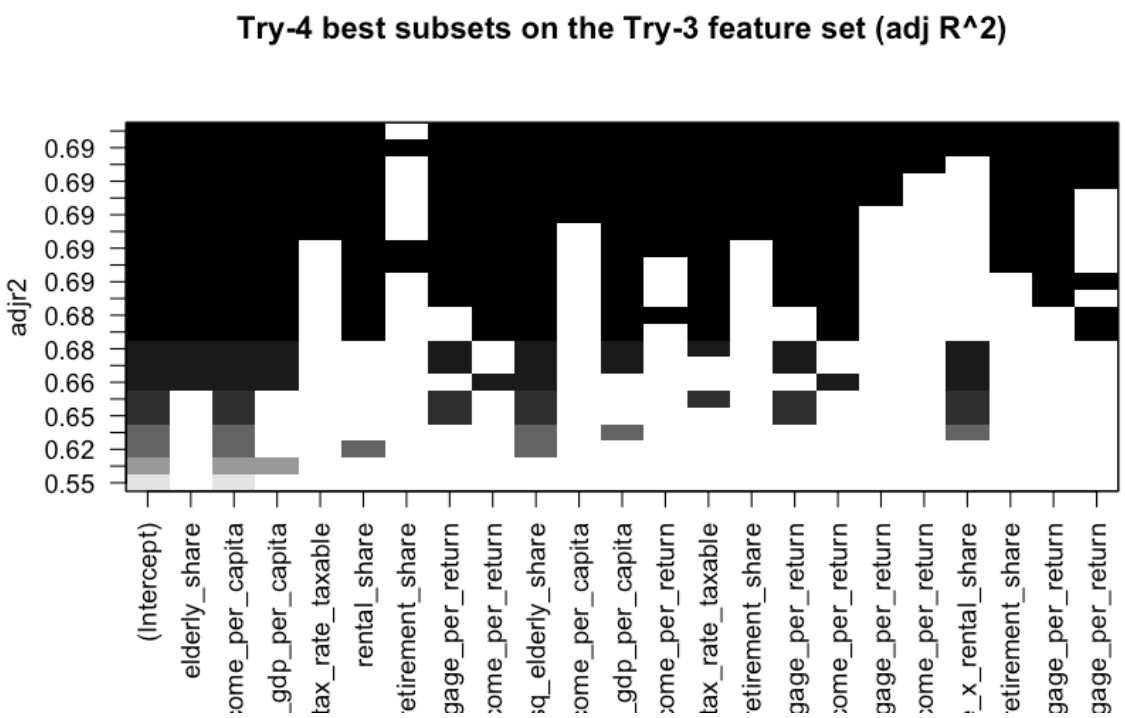
```
## Design matrix: 2850 rows x 22 features
```

```
nvmax_final <- ncol(mm)
```

```
rs_final2 <- regsubsets(response_var ~ ., data = bss_df,
                        nvmax = nvmax_final, nbest = 1,
                        method = "exhaustive", really.big = TRUE)

ss2 <- summary(rs_final2)

cat("Adj R^2 by size:\n"); print(round(ss2$adjr2, 4))
```

```
## Adj R^2 by size:

##  [1] 0.5502 0.5901 0.6201 0.6320 0.6505 0.6580 0.6642 0.6712 0.6772 0.6811
## [11] 0.6830 0.6849 0.6868 0.6889 0.6905 0.6913 0.6921 0.6928 0.6933 0.6936
## [21] 0.6939 0.6938
```

```
plot(rs_final2, scale = "adjr2",
     main = "Try-4 best subsets on the Try-3 feature set (adj R^2)")
```




```
size_final <- which.max(ss2$adjr2)
final_best_cnames <- setdiff(names(coef(rs_final2, size_final)), "(Intercept)")
final_best_terms <- unname(name_map[final_best_cnames])

cat("Chosen size (argmax adj R^2):", size_final, "\n")
```

```
## Chosen size (argmax adj R^2): 21
```

```r
cat("Final-best features:\n", paste(final_best_terms, collapse = ", "), "\n", sep = "")
```

```
## Final-best features:
## elderly_share, log_income_per_capita, log_gdp_per_capita, eff_tax_rate_taxable, rental_share, log_mo
```

```r
model_final_best <- lm(as.formula(paste("response_var ~",
                    paste(final_best_cnames, collapse = " + "))), data = bss_df)

cat("\nTry-4 (final-best) adj R^2:",
    round(summary(model_final_best)$adj.r.squared, 4), "\n")
```

```
##
## Try-4 (final-best) adj R^2: 0.6939
```

```r
summary(model_final_best)
```

```
##
## Call:
## lm(formula = as.formula(paste("response_var ~", paste(final_best_cnames,
##     collapse = " + "))), data = bss_df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.69260 -0.25771 -0.03366  0.23480  1.82597
##
## Coefficients:
##                                                  Estimate Std. Error
## (Intercept)                                    -2.314033   1.997542
## elderly_share                                 -28.818448   3.284301
## log_income_per_capita                           5.100834   0.509685
## log_gdp_per_capita                             -0.608999   0.096438
## eff_tax_rate_taxable                           30.141786   7.980208
## rental_share                                   -5.522687   1.991147
## log_mortgage_per_return                        -0.274351   0.036328
## log_salt_income_per_return                     -0.164472   0.033180
## sq_elderly_share                               30.674507   2.846633
## elderly_share_x_log_income_per_capita           2.797516   1.011381
## elderly_share_x_log_gdp_per_capita              2.001505   0.494230
## elderly_share_x_log_salt_income_per_return     -0.132242   0.031048
## log_income_per_capita_x_eff_tax_rate_taxable   -9.440191   1.999025
## log_income_per_capita_x_retirement_share       -3.313678   0.736444
## log_income_per_capita_x_log_mortgage_per_return  0.085790   0.010896
## log_income_per_capita_x_log_salt_income_per_return  0.040406   0.010389
## log_gdp_per_capita_x_log_mortgage_per_return   -0.014947   0.004353
## log_gdp_per_capita_x_log_salt_income_per_return  0.009083   0.003988
## eff_tax_rate_taxable_x_rental_share            13.987948   7.498828
## eff_tax_rate_taxable_x_retirement_share        57.132357  13.098971
## rental_share_x_log_mortgage_per_return         -0.099987   0.026950
## retirement_share_x_log_mortgage_per_return      0.089057   0.041225
##                                                t value Pr(>|t|)
## (Intercept)                                     -1.158 0.246782
```

```
## elderly_share                                        -8.775  < 2e-16 ***
## log_income_per_capita                                10.008  < 2e-16 ***
## log_gdp_per_capita                                   -6.315 3.13e-10 ***
## eff_tax_rate_taxable                                  3.777 0.000162 ***
## rental_share                                         -2.774 0.005580 **
## log_mortgage_per_return                              -7.552 5.75e-14 ***
## log_salt_income_per_return                           -4.957 7.58e-07 ***
## sq_elderly_share                                     10.776  < 2e-16 ***
## elderly_share_x_log_income_per_capita                 2.766 0.005711 **
## elderly_share_x_log_gdp_per_capita                    4.050 5.27e-05 ***
## elderly_share_x_log_salt_income_per_return           -4.259 2.12e-05 ***
## log_income_per_capita_x_eff_tax_rate_taxable         -4.722 2.44e-06 ***
## log_income_per_capita_x_retirement_share             -4.500 7.08e-06 ***
## log_income_per_capita_x_log_mortgage_per_return       7.874 4.86e-15 ***
## log_income_per_capita_x_log_salt_income_per_return    3.889 0.000103 ***
## log_gdp_per_capita_x_log_mortgage_per_return         -3.434 0.000604 ***
## log_gdp_per_capita_x_log_salt_income_per_return       2.278 0.022810 *
## eff_tax_rate_taxable_x_rental_share                   1.865 0.062236 .
## eff_tax_rate_taxable_x_retirement_share               4.362 1.34e-05 ***
## rental_share_x_log_mortgage_per_return               -3.710 0.000211 ***
## retirement_share_x_log_mortgage_per_return            2.160 0.030836 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4374 on 2828 degrees of freedom
## Multiple R-squared:  0.6961, Adjusted R-squared:  0.6939
## F-statistic: 308.5 on 21 and 2828 DF,  p-value: < 2.2e-16
```

```
# Side-by-side fit comparison: Try-0 through Try-4

fit_progression <- data.frame(
  Stage   = c("Try-0 (untransformed top)",
              "Try-1 (logs + quad)",
              "Try-2 (all pairwise interactions)",
              "Try-3 (Try-2 sig interactions)",
              "Try-4 (BSS over Try-3 features)"),
  AdjR2   = round(c(summary(model_top)$adj.r.squared,
                    summary(model_try1)$adj.r.squared,
                    summary(model_interaction)$adj.r.squared,
                    summary(model_final)$adj.r.squared,
                    summary(model_final_best)$adj.r.squared), 4),
  NFeatures = c(length(top_vars),
                length(coef(model_try1)) - 1,
                length(coef(model_interaction)) - 1,
                length(coef(model_final)) - 1,
                length(coef(model_final_best)) - 1))

fit_progression
```

```
##                              Stage  AdjR2 NFeatures
## 1         Try-0 (untransformed top) 0.4943         8
## 2               Try-1 (logs + quad) 0.6531         9
## 3 Try-2 (all pairwise interactions) 0.6958        37
## 4    Try-3 (Try-2 sig interactions) 0.6938        22
```

```
## 5   Try-4 (BSS over Try-3 features) 0.6939        21
```

The reason for dropping so many variables between try 2 through 4 was for the sake of parsimony.

10 Final Model Diagnostics

All diagnostics below are computed on the Try-4 final model (`model_final_best`), which is the best-subsets pick over the Try-3 feature set.

```
par(mfrow = c(2, 2))
plot(model_final_best, cex = 0.4, col = adjustcolor("steelblue", 0.5))
```

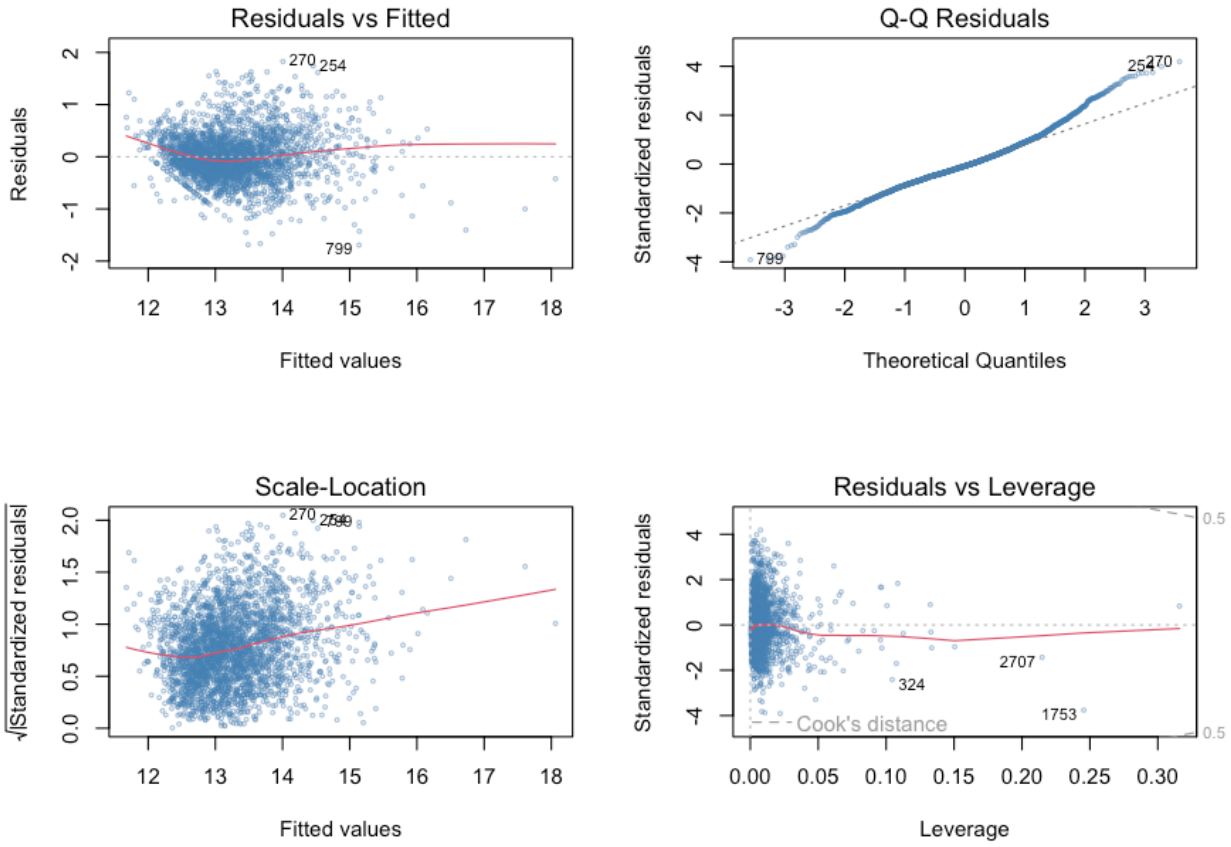


Figure 7: Diagnostic plots for the Try-4 final model. Residuals are approximately centered with mild heteroskedasticity, addressed by the HC1 robust standard errors below.

```
par(mfrow = c(1, 1))
```

```
coeftest(model_final_best, vcov = vcovHC(model_final_best, type = "HC1"))
```

```
##
```

```
## t test of coefficients:
##
##                                                    Estimate  Std. Error
## (Intercept)                                      -2.3140326   2.5191089
## elderly_share                                   -28.8184477   4.7409578
## log_income_per_capita                             5.1008342   0.6607335
## log_gdp_per_capita                               -0.6089992   0.1458423
## eff_tax_rate_taxable                             30.1417857   9.4739695
## rental_share                                     -5.5226870   2.1273399
## log_mortgage_per_return                          -0.2743512   0.0436497
## log_salt_income_per_return                       -0.1644722   0.0390472
## sq_elderly_share                                 30.6745074   3.6425785
## elderly_share_x_log_income_per_capita             2.7975164   1.5275931
## elderly_share_x_log_gdp_per_capita                2.0015048   0.8041653
## elderly_share_x_log_salt_income_per_return       -0.1322420   0.0395459
## log_income_per_capita_x_eff_tax_rate_taxable     -9.4401909   2.4218446
## log_income_per_capita_x_retirement_share         -3.3136783   0.8595589
## log_income_per_capita_x_log_mortgage_per_return   0.0857896   0.0132291
## log_income_per_capita_x_log_salt_income_per_return  0.0404062  0.0126315
## log_gdp_per_capita_x_log_mortgage_per_return     -0.0149471   0.0054899
## log_gdp_per_capita_x_log_salt_income_per_return   0.0090833   0.0051906
## eff_tax_rate_taxable_x_rental_share              13.9879482   7.9986652
## eff_tax_rate_taxable_x_retirement_share          57.1323571  14.9341335
## rental_share_x_log_mortgage_per_return           -0.0999873   0.0273173
## retirement_share_x_log_mortgage_per_return        0.0890570   0.0431864
##                                                  t value  Pr(>|t|)
## (Intercept)                                      -0.9186 0.3583875
## elderly_share                                    -6.0786 1.375e-09 ***
## log_income_per_capita                             7.7200 1.603e-14 ***
## log_gdp_per_capita                               -4.1757 3.060e-05 ***
## eff_tax_rate_taxable                              3.1815 0.0014808 **
## rental_share                                     -2.5961 0.0094790 **
## log_mortgage_per_return                          -6.2853 3.777e-10 ***
## log_salt_income_per_return                       -4.2121 2.609e-05 ***
## sq_elderly_share                                  8.4211 < 2.2e-16 ***
## elderly_share_x_log_income_per_capita             1.8313 0.0671575 .
## elderly_share_x_log_gdp_per_capita                2.4889 0.0128702 *
## elderly_share_x_log_salt_income_per_return       -3.3440 0.0008365 ***
## log_income_per_capita_x_eff_tax_rate_taxable     -3.8979 9.927e-05 ***
## log_income_per_capita_x_retirement_share         -3.8551 0.0001183 ***
## log_income_per_capita_x_log_mortgage_per_return   6.4849 1.044e-10 ***
## log_income_per_capita_x_log_salt_income_per_return  3.1988 0.0013951 **
## log_gdp_per_capita_x_log_mortgage_per_return     -2.7227 0.0065157 **
## log_gdp_per_capita_x_log_salt_income_per_return   1.7499 0.0802359 .
## eff_tax_rate_taxable_x_rental_share               1.7488 0.0804366 .
## eff_tax_rate_taxable_x_retirement_share           3.8256 0.0001333 ***
## rental_share_x_log_mortgage_per_return           -3.6602 0.0002566 ***
## retirement_share_x_log_mortgage_per_return        2.0622 0.0392842 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
ali <- alias(model_final_best)
if (!is.null(ali$Complete) && nrow(ali$Complete) > 0) {
  aliased_terms <- rownames(ali$Complete)
```

```
  cat("Aliased terms (dropped before VIF):\n  ",
      paste(aliased_terms, collapse = ", "), "\n", sep = "")
  rhs_for_vif <- setdiff(final_best_cnames, aliased_terms)
  fit_for_vif <- lm(as.formula(paste("response_var ~",
                                     paste(rhs_for_vif, collapse = " + "))),
                    data = bss_df)
  print(vif(fit_for_vif))
} else {
  print(vif(model_final_best))
}
```

```
##                                       elderly_share
##                                            322.638075
##                                 log_income_per_capita
##                                            184.140483
##                                    log_gdp_per_capita
##                                             30.082586
##                                  eff_tax_rate_taxable
##                                            323.926387
##                                         rental_share
##                                            138.624245
##                                log_mortgage_per_return
##                                            784.164050
##                            log_salt_income_per_return
##                                            663.346035
##                                      sq_elderly_share
##                                             44.622889
##                 elderly_share_x_log_income_per_capita
##                                            539.508457
##                    elderly_share_x_log_gdp_per_capita
##                                            111.932268
##            elderly_share_x_log_salt_income_per_return
##                                             27.021434
##          log_income_per_capita_x_eff_tax_rate_taxable
##                                            600.715007
##              log_income_per_capita_x_retirement_share
##                                            132.365016
##       log_income_per_capita_x_log_mortgage_per_return
##                                           1115.352691
## log_income_per_capita_x_log_salt_income_per_return
##                                           1037.353409
##          log_gdp_per_capita_x_log_mortgage_per_return
##                                            157.884821
##       log_gdp_per_capita_x_log_salt_income_per_return
##                                            138.459244
##                   eff_tax_rate_taxable_x_rental_share
##                                            135.231253
##               eff_tax_rate_taxable_x_retirement_share
##                                            131.497148
##                rental_share_x_log_mortgage_per_return
##                                              2.737602
##            retirement_share_x_log_mortgage_per_return
##                                              4.201485
```

11 County Maps

All maps below use the same red-white-blue diverging palette so that patterns can be compared directly across panels. Each variable is centered on its own median: red counties are below the typical county on that variable and blue counties are above. Centering on the median (rather than on zero) is what produces a balanced red-to-blue contrast in the change maps; if I centered on zero instead, the response map would be almost entirely blue because roughly 95 percent of counties grew their stub-8 population over this window. Tail values are quantile-clipped at the 2nd and 98th percentiles so a handful of extreme counties do not dominate the color scale.

```
counties_sf <- counties(cb = TRUE, year = 2023)

df_map <- df %>% mutate(fips = sprintf("%05d", as.numeric(fips)))
map_sf <- counties_sf %>%
  left_join(df_map, by = c("GEOID" = "fips"))
```

```
plot_map <- function(sf_obj, fill_var, title,
                     midpoint = NA, clip_q = c(0.02, 0.98)) {
  vals <- sf_obj[[fill_var]]
  qs   <- quantile(vals, probs = clip_q, na.rm = TRUE)
  mid  <- if (is.na(midpoint)) median(vals, na.rm = TRUE) else midpoint

  ggplot(sf_obj) +
    geom_sf(aes(fill = .data[[fill_var]]), color = NA) +
    coord_sf(xlim = c(-125, -66), ylim = c(25, 50), expand = FALSE) +
    labs(title = title, fill = NULL) +
    theme_void(base_size = 9) +
    theme(plot.title = element_text(size = 9),
          legend.key.height = unit(0.4, "cm"),
          legend.key.width  = unit(0.3, "cm"),
          legend.text = element_text(size = 7)) +
    scale_fill_gradient2(low = "darkred", mid = "white", high = "darkblue",
                         midpoint = mid,
                         limits = qs, oob = scales::squish,
                         na.value = "grey90")
}
```

11.1 Response variable and key predictors

```
m_resp  <- plot_map(map_sf, "chg_returns_per_1k",
                    "Change in stub-8 returns / 1k pop, 2019 to 2021")
m_work <- plot_map(map_sf, "working_age_share", "Working-age share (2022)")
m_eld <- plot_map(map_sf, "elderly_share", "Elderly share (2022)")
m_labor <- plot_map(map_sf, "labor_share", "Labor income share (stub 8, 2022)")
m_tax <- plot_map(map_sf, "tax_per_return","Tax per return (stub 8, 2022)")
m_inc <- plot_map(map_sf, "income_per_capita", "Income per capita (2022)")
m_mort <- plot_map(map_sf, "mortgage_per_return",
                    "Mortgage interest per return (stub 8, 2022)")

(m_resp | m_work) / (m_eld | m_labor)
```

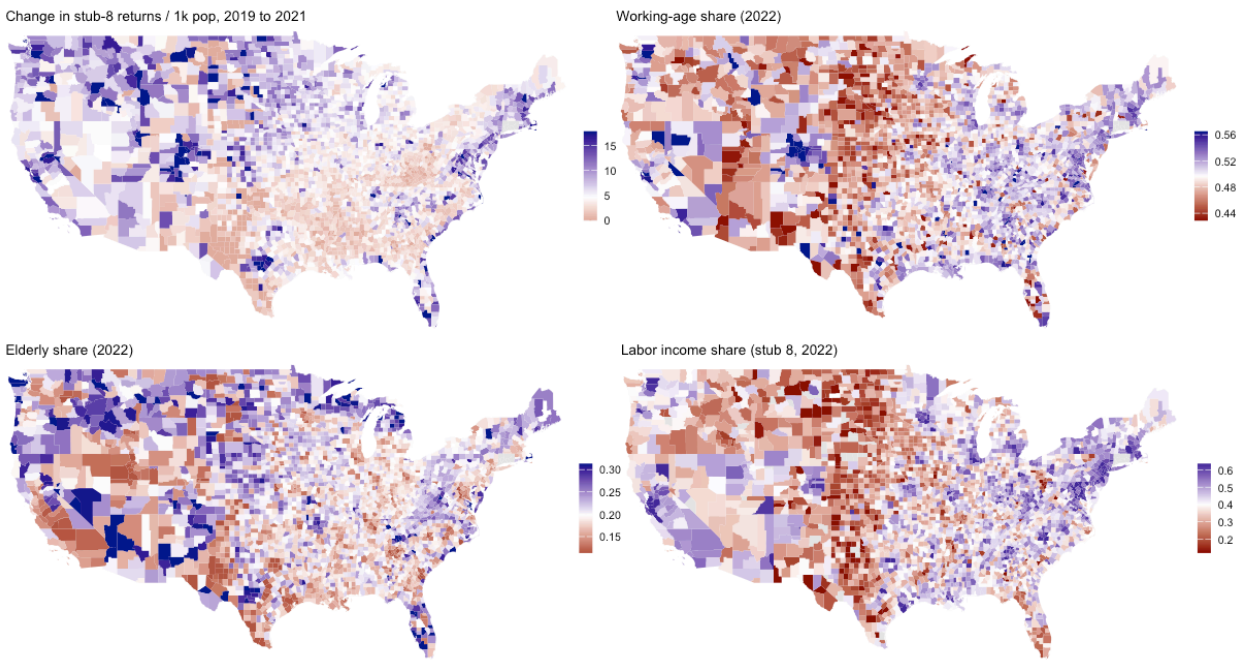


Figure 8: Geographic distribution of the response and four key predictors. The Sun Belt, Mountain West, and Texas / Florida metros show above-median top-bracket growth (blue); the rural Plains, parts of the Midwest, and pockets of the Deep South are below median (red).

```
(m_resp | m_tax)  / (m_mort | m_inc)
```

A few observations from the maps. The response map (top-left in each pair) shows two clear gradients. The first is the well-known Sun Belt and Mountain West story: Texas, Florida, the Carolinas, Tennessee, Arizona, Idaho, Utah, and Colorado all appear in blue. The second is more subtle and worth flagging: the entire urban-coastal corridor from Boston through DC also appears blue, even though that corridor lost net population during the pandemic. Both patterns are visible because the response is per capita, so a county can shrink in absolute population while still adding stub-8 filers per 1,000 residents.

The predictor maps line up with the response map in instructive ways. Income per capita and tax per return both light up the same coastal-metro corridor that is blue on the response map; mortgage interest per return picks out the most expensive housing markets specifically (the Bay Area, NYC suburbs, Boston, DC, parts of Florida). The elderly-share map is the most interesting one: very-high-elderly counties in Florida and Arizona are blue on both the elderly-share map and on the response, which is the equity-rally capital-gains channel for retirees, exactly the mechanism that motivated the quadratic term on elderly share. The working-age and labor-income-share maps are the noisiest of the six; both vary more by region than by metro, and their relationship to the response shows up more clearly in the regression than in the geography.

11.2 Individuals response

```
m_indiv <- plot_map(map_sf, "chg_individuals_per_1k",
                    "Change in stub-8 individuals / 1k pop, 2019 to 2021")

m_indiv | m_work | m_eld
```

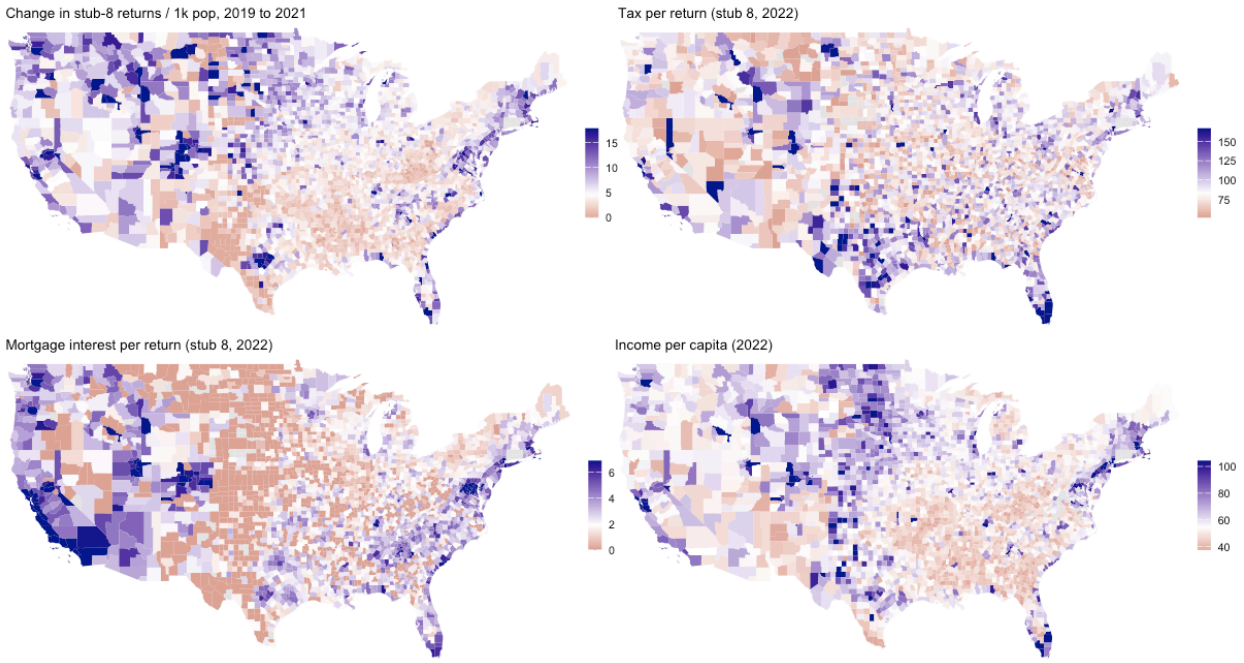


Figure 9: Geographic distribution of the response and four key predictors. The Sun Belt, Mountain West, and Texas / Florida metros show above-median top-bracket growth (blue); the rural Plains, parts of the Midwest, and pockets of the Deep South are below median (red).

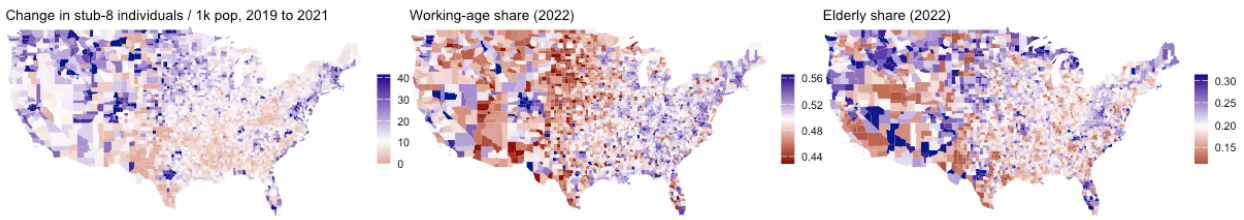


Figure 10: The change in stub-8 individuals (left) traces nearly the same geography as the change in returns (top of the previous figure). The two responses move together because one return covers one or two individuals.

12 Limitations

The 2019 to 2021 window spans the COVID-19 pandemic shock, which makes external validity uncertain. Remote-work flexibility enabled a relocation wave that would not have happened under normal conditions, so the geographic patterns may be partly pandemic-specific. Repeating this exercise on a pre-pandemic window (e.g., 2015 to 2017) would help separate structural patterns from transitory ones.

One technical limitation worth flagging is that the `na.omit` step in the candidate-pool construction drops counties that are missing any of the numeric predictors, reducing the working sample significantly. This is a wide-data version of the IRS suppression rule (which removes county-stub cells with fewer than 20 returns) and biases the working sample toward larger, more data-rich counties.

Additionally, the IRS migration variables (net migration filers and persons, inflow and outflow filers and AGI, migration rates) are excluded from the candidate pool because they are themselves constructed from the same year-over-year filer counts that produce the response. Letting them in would inflate adj R-squared without adding economic content. The county-to-county SOI migration files at the stub-8 level (not used here) would be the right way to bring a genuine migration channel into the model.

13 Conclusions

Summary of main findings:

The final model (Try-4, the best-subsets pick over the Try-3 feature set) reaches an adjusted R-squared of 0.6939 on 21 features. Twenty-one out of the original ~40 candidate predictors and their pairwise interactions are enough to explain about 69% of the cross-county variation in top-bracket filer growth.

The strongest single predictors of stub-8 growth are county income per capita and AGI per person on the income side, average mortgage interest and SALT income tax deductions on the housing-and-taxation side, and the elderly and working-age population shares on the demographic side. Male share, labor income share, and transfer share enter mostly through interactions with income, not as standalone main effects.

Box-Cox transforming the response (optimal lambda around 0.6) and trimming about 112 high-leverage counties via Cook's distance lift the model from adjusted R-squared 0.57 to 0.69, a clean +0.12 gain. EITC and the refundable child and education credits are excluded from the candidate pool because they are structurally zero at AGI stub 8 by design. The IRS migration variables (net migration, inflow, outflow, migration rate) are excluded because they are themselves derived from the same filer counts that produce the response; including them would inflate the fit inaccurately, rather than through real economic content.

The two-year national growth in stub-8 returns is roughly 28%, far too large to be migration in any conventional sense. The data is most consistent with bracket creep, capital-gains realizations during the 2020 to 2021 equity-market rally, and a smaller share of genuine residential mobility. The model identifies where these forces concentrate geographically (the Sun Belt, the Mountain West, the Texas and Florida metros, and the coastal-corridor professional hubs).

14 Final thoughts and Extension of Research

Closing thoughts:

A small and economically coherent set of county characteristics, when combined with a Box-Cox transformation of the response and an outlier-trimmed estimation sample, explains a substantial share of the variation in top-bracket filer growth between 2019 and 2021. The final best-subsets model reaches an adjusted R-squared of 0.6939, which is a strong result for a cross-sectional county-level regression and well above the ceiling of any specification that ignored the methodological details. The model's predictive power comes mostly from the threshold-crossing mechanism: counties whose income distribution sits closer to the

$200,000 stub-8 boundary produce more new top-bracket filers per unit of nominal income growth, and the variables that pick this up most cleanly (income per capita, AGI per person, the deduction proxies for housing-cost exposure) are exactly the ones the best-subsets pipeline keeps in every iteration.

The next step would be to repeat the same pipeline on the lower AGI stubs and compare coefficients across the income distribution. Top-bracket filers respond to a relatively coherent set of forces including income growth, capital gains, and expensive-housing geography, but those same forces work very differently for stubs 1 through 4, where housing affordability, transfer eligibility, and job access dominate.